
Welcome Offer Access to everything. Now 30% off. [Upgrade now](#)

Level Up Coding

You're reading for free via [Fareed Khan's](#) Friend Link.

★ Member-only story

AI +

Artificial Intelligence ✓

Data Science ✓

Machine Learning ✓

Python ✓

Building Kimi K3 2.8T Model in C to Run on 8GB RAM

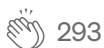
Stream the model from disk instead of holding it in memory



Fareed Khan

Follow

94 min read · 1 day ago



293



7



1



Read this story for free: [link](#)

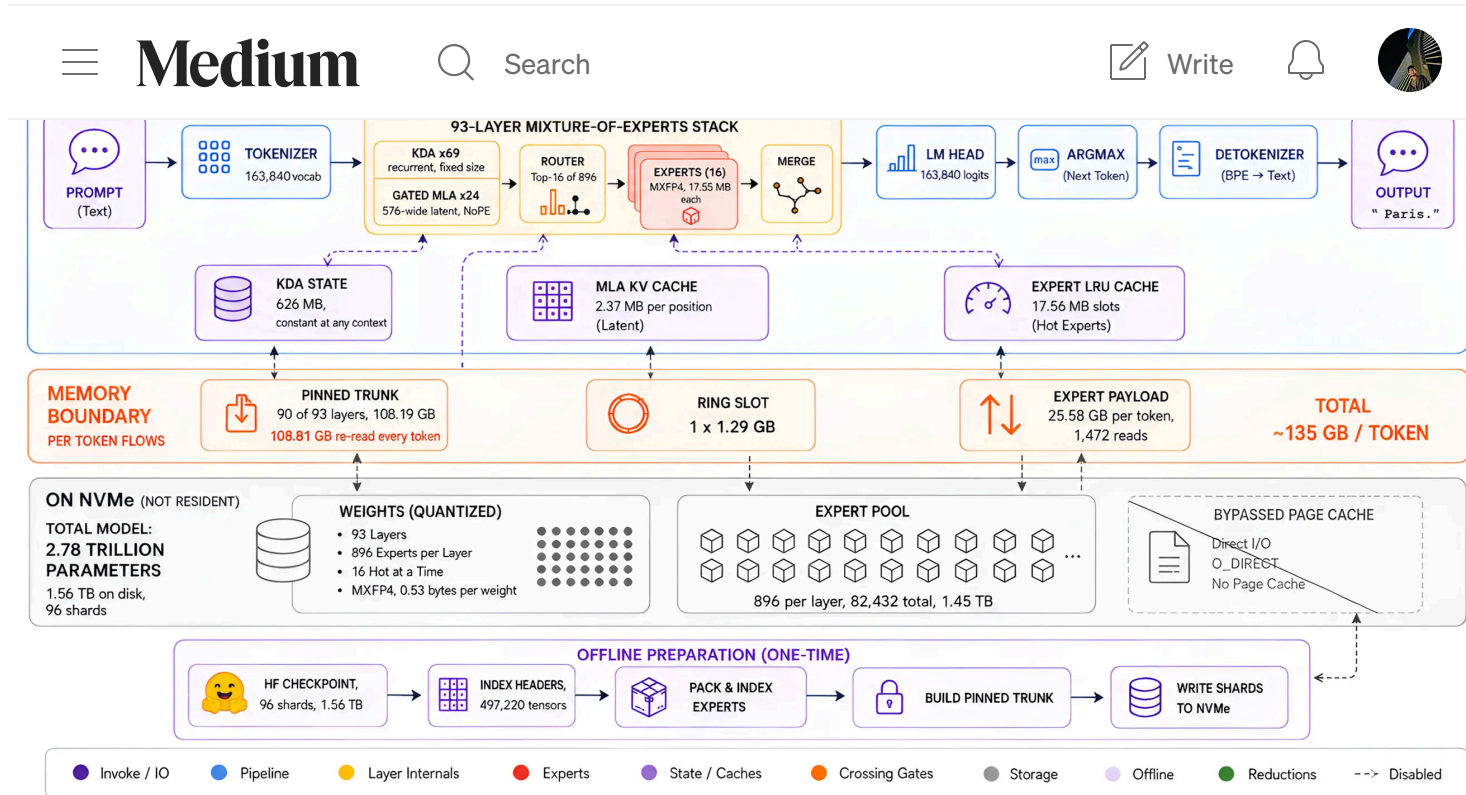
Recently I deployed Kimi K3 on 32 H100 GPUs for my company, but almost nobody has a cluster. Kimi K3 has **2.78 trillion parameters** and is **1.56 terabytes** even as shipped, so no consumer machine can dream of holding it, and waiting longer does not help, because the wall is not speed, it is capacity. But it is a Mixture of Experts, so only 16 of its 896 experts per layer fire for

any given token and the rest sit asleep on disk. C has a format advantage here. Keep the always on part in memory, stream the sleeping experts, and it fits in **8.24 gigabytes** on one CPU with no GPU, running an engine we build from scratch in pure C.



KIMI K3 IN C ENGINE

Open in app ↗

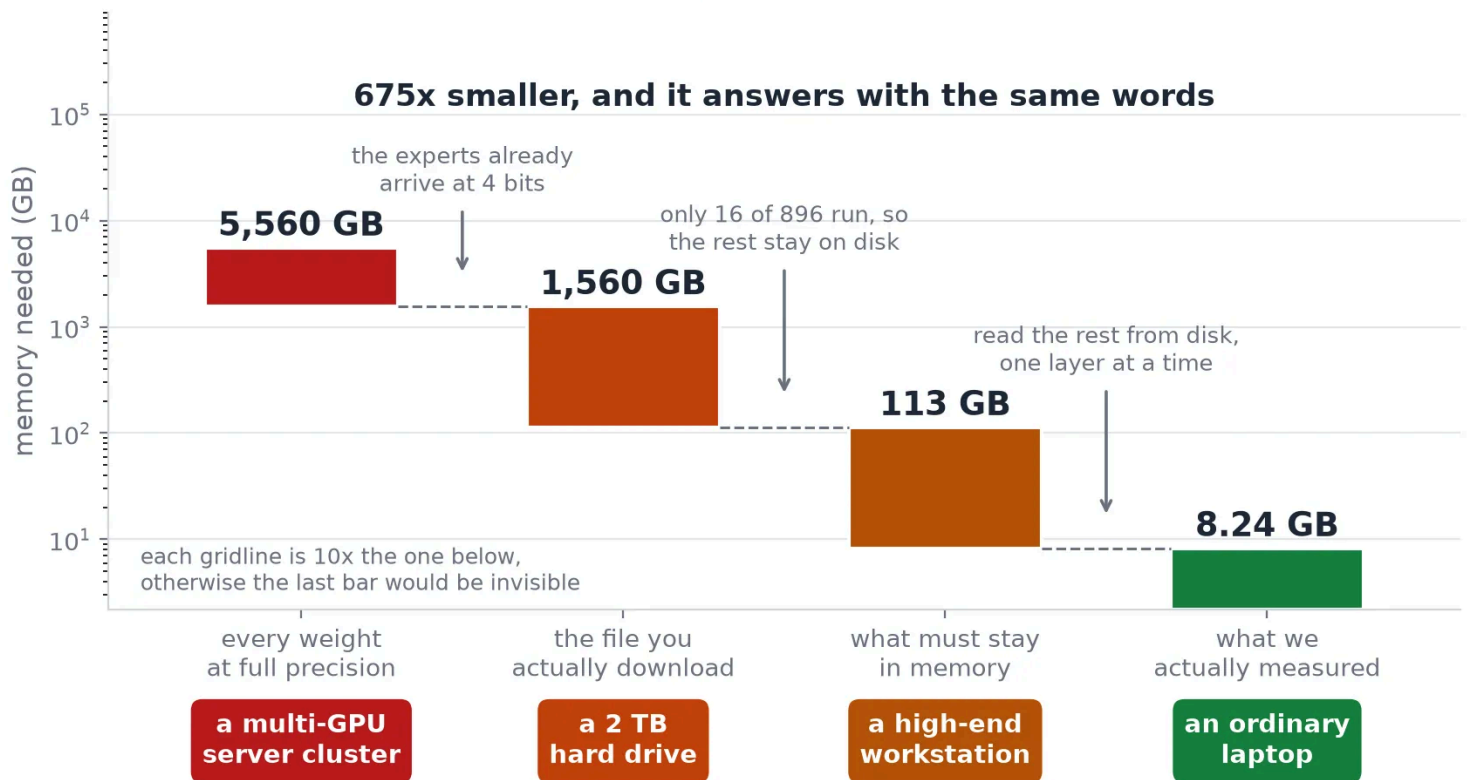


Optimized Architecture of Kimi K3 in C to make it fit on consumer hardware (Created by [Fareed Khan](#))

By the end of this post we will have built every box in that diagram from scratch, one component at a time, and run prompts through the finished engine on a single CPU.

Here is what all of that buys us, and it is the one number the rest of the post exists to earn.

How a model far too big for any desk ends up running on one



Four steps from a server cluster down to an ordinary laptop, with the same output at both ends (Created by Fareed Khan)

Here is everything we build, six C files and two headers with no BLAS and no framework, top to bottom, one component at a time:

- **Count the bytes honestly:** how far we have to travel, and which parts of the model are in the way.
- **Read the checkpoint:** index 497,220 tensors across 96 shards in under a second, without loading any of them.
- **Refuse to guess:** a config reader and a tokenizer that fail loudly, because the danger is a fluent model of the wrong architecture.
- **Reduction one, the format:** the experts already ship at 0.53125 bytes per weight, and we multiply straight from the packed nibbles.

- **Reduction two, KDA:** 69 of the 93 layers carry a recurrent state of 626 megabytes, whatever the context length.
- **Reduction three, MLA:** the other 24 cache one shared latent instead of per head keys and values, 53 times smaller for identical math.
- **Reduction four, streaming:** pack the trunk into 93 contiguous runs and read one layer at a time, turning the last floor into a dial.
- **Prove it is Kimi K3:** four levels of validation, ending with all 163,840 logits checked against a PyTorch reference.
- **Generate text:** four decoded prompts, including a correct recursive Fibonacci and the model describing its own routing accurately.
- **Measure it, then measure the measurements:** twelve budgets, a cache that does nothing, an allocation rule worth 1.69x, and a 33 percent noise floor.

Every number and every log line in this post comes from runs on our own hardware. All of the code is available in my GitHub repository (theory plus code):

GitHub - FareedKhan-dev/kimi-k3-in-c: A 2.78-trillion-parameter Kimi K3 running inference on a...

A 2.78-trillion-parameter Kimi K3 running inference on a single CPU in 8.24 GB of RAM. Portable C99: no BLAS, no...

github.com

The codebase is organized as follows.

```

kimi-k3-in-c/
├── include/k3/
│   ├── k3.h           # the public header: config, weights, every kernel pro
│   └── k3_cfg.h       # config reader, header-only, refuses to substitute de
├── src/
│   ├── core/k3_ops.c  # every numeric kernel: RMSNorm, KDA, MLA, MoE, MXFP4
│   ├── io/k3_st.c     # safetensors reader, hand-written JSON scan, O_DIRECT
│   ├── io/k3_load.c   # locating one expert's bytes inside a shard
│   ├── io/k3_trunk.c  # streaming the dense trunk, pinned prefix plus a ring
│   ├── cache/k3_cache.c # the routed-expert LRU cache and its batch prefetch
│   ├── model/k3_bind.c # binding checkpoint tensor names to kernel arguments
│   ├── tokenizer/k3_tok.h # byte-level BPE loaded from tiktoken.model
│   └── cli/k3_run.c    # the k3 binary: memory plan, decode loop, reporting
├── tools/             # python: pack the trunk, replay the cache, verify aga
├── benchmarks/        # the cgroup memory ladder and the split sweep
└── tests/             # fixtures, the tiny oracle, the 93-layer conformance

```

So let us get started and build it up, one piece at a time.

Table of Contents

- The Model Does Not Fit, and by How Much
- The Machine, and What It Is Allowed to Assume
- One Small C Codebase, No BLAS, No Framework
- Reading a 1.56 TB Checkpoint From Its Headers
- The Config Reader That Refuses to Guess
- The Tokenizer, Byte for Byte
- Reduction One: the Experts Already Ship at Half a Byte
- Kernels With a Floating Point Contract
- Reduction Two: KDA, Attention With a Memory That Never Grows

- Reduction Three: MLA, One Latent Instead of Ninety-Six Heads
- Attention Residuals: Layers That Look Back
- Picking 16 Experts of 896
- Packing the Trunk: 93 Layers, One Read Each
- Reduction Four: Streaming the Trunk Turns a Floor Into a Dial
- An LRU Cache for the Experts
- How Big Should That Cache Be? Ask the Trace
- Proving It: a Tiny Oracle First
- Proving It on the Full Checkpoint
- The First Tokens
- Sustained Generation: Text In, Text Out
- The Memory Ladder: 8 GB to 224 GB
- The Cache That Was Not Participating
- Allocation Beats Capacity
- Measuring the Measurement
- Storage Is the Whole Game
- Why We Do Not Quantize the Trunk
- What This Engine Does Not Do
- Running It End to End

The Model Does Not Fit, and by How Much

Before writing any code, I want to know exactly what we are up against. Vague statements like “it is too big” are useless, because the whole job is to find which specific bytes we can avoid holding.

The naive requirement is the one every parameter count implies.

$$\text{bytes} = P \cdot b = 2.78 \times 10^{12} \cdot 2 = 5.56 \text{ TB}$$

The naive requirement: every parameter resident at bf16

The naive requirement: every parameter resident at bf16 (Created by [Fareed Khan](#))

So 5.56 terabytes is the number to beat. The first thing that helps is that Kimi K3 is a **mixture of experts**, which means the model has 896 separate expert networks in each of most layers, and a small router picks only a handful of them for each token.

One token wakes 16 experts and leaves 880 asleep (Created by [Fareed Khan](#))

Kimi K3 has 93 layers. Layer 0 is a plain dense feed-forward layer, so the other 92 layers route, and each of those picks the top 16 experts out of 896.

Only 16 of 896 experts fire per layer, so most of the model sleeps (Created by [Fareed Khan](#))

```
=== shard census: what the 1.56 TB actually is ===
note      : the checkpoint is NOT downloaded locally. These are the published size
            the tensor->shard index, which is why a 1.56 TB model can be inventoried
            exactly from 137 MB of headers.

shards          : 96
total bytes     : 1560936091448  (1.56 TB)

--- routed experts (the part that is streamed, never resident) ---
experts total   : 82,432  (896 routed x 92 MoE layers)
bytes per expert : 17,547,264  exactly
                 = 33,030,144 params x 0.53125 bytes
                 = 0.5 bytes/nibble + 1/32 byte for the shared E8M0 scale
routed expert set : 82,432 x 17,547,264 = 1.447 TB
```

About 104 billion parameters are active for any given token, out of 2.78 trillion. That is roughly 3.7 percent. The other 96.3 percent still has to exist somewhere we can reach, but it does not have to be in RAM.

Now let us count the actual bytes on disk rather than guessing. Our capture tool read the published shard index and the per-shard sizes, and worked out what the 1.56 terabytes is made of. Here is what it reported.

There are **82,432 routed experts** in total, and each one occupies exactly **17,547,264 bytes**. Multiply those together and the routed experts alone are **1.447 terabytes**, which is 93 percent of the entire checkpoint. Everything else, the attention projections, the routers, the norms, the embeddings, is the remaining 7 percent.

Where the 1.56 TB lives: 93% of it is experts that never load (Created by [Fareed Khan](#))

That census is the whole strategy in one picture. If we can arrange for those 1.447 terabytes to be reachable but never resident, the memory problem shrinks by more than an order of magnitude before we have written a single kernel.

What is left over when the experts are excluded is the part that runs on every single token no matter what the router decides.

The always-active set: 113.49 GB at bfloat16, everything else is streamable (Created by [Fareed Khan](#))

That is **56,743,648,000 parameters**, or 113.49 gigabytes at bfloat16. Of that, 108.81 gigabytes is the per-layer dense trunk and 4.70 gigabytes is the embedding table plus the output head.

So here is the ledger we are going to work down, and I will restate one line of it every time we earn it.

- **5,560 GB** is every parameter at bfloat16, which is where we start.
- **1,560 GB** is the checkpoint as shipped, because the experts already arrive at half a byte per weight.
- **113.49 GB** is what has to be resident once routing means the experts never load.
- **8.24 GB** is what we actually measured, once the trunk is streamed instead of held.

End to end that is a **675x reduction** from the bfloat16 model and **189x** from the shipped checkpoint. Nothing is approximated and no weight is dropped, because the output at the bottom of that ladder is byte for byte the output at the top.

Four numbers, four sections. Let us go and earn each one.

The Machine, and What It Is Allowed to Assume

Everything in this post was measured on one workstation, and I want to put its specification on the table now so that no number later on is mysterious.

It is a two-socket AMD EPYC 7763 box with 124 cores and no simultaneous multithreading, 228 gigabytes of RAM, and 3.2 terabytes of NVMe. It also has four NVIDIA L40 GPUs, and they sat completely idle for the entire campaign, because this engine has no GPU path at all. Here is the relevant part of the environment capture.

```
--- ISA (note: AVX2 present, AVX-512 ABSENT) ---  
avx avx2 fma sse4_2
```

```
--- memory ---
```

Mem:	228Gi	5.1Gi	207Gi	3.1Mi	18Gi	223Gi
MemTotal:	239308464 kB					
MemAvailable:	233961008 kB					
Hugepagesize:	2048 kB					

Note that there is **no AVX-512**. The engine needs AVX2 and FMA and nothing more, which is exactly the instruction set on any desktop CPU from the last decade.

The storage numbers matter more than the CPU numbers, and one of them runs against the usual expectation.

```
--- storage bandwidth, measured ---  
O_DIRECT cold : 3.2 GB/s      (dd bs=4M iflag=direct after drop_caches)  
buffered warm : 2.3 GB/s  
engine, trunk : 5373-6064 MB/s sustained during real runs  
NOTE O_DIRECT is FASTER than buffered here. That is the opposite of the usual  
expectation, and it is why the engine opens the trunk O_DIRECT.
```

Reading with `O_DIRECT`, which bypasses the operating system page cache entirely, is **faster** here than reading through the cache. That is backwards from what most people expect, and it is the single measurement that decided the whole I/O design. We open the trunk with `O_DIRECT` because on this device the page cache is pure overhead.

One binary, four kinds of machine, and one identical answer (Created by [Fareed Khan](#))

One piece of hygiene is worth stating, because a machine this loaded is easy to measure badly.

```
--- measurement hygiene ---  
unattended-upgrades: STOPPED and DISABLED before measurement (was using ~63% of  
  core during the smoke run).  
apt-daily.timer and apt-daily-upgrade.timer: DISABLED
```

A background package updater eating most of a core moves a timing by more than most optimisations do, so it goes off before anything is measured.

Now, how much memory does the engine actually need? The tempting approach is to multiply the config values by hand, and that gives the wrong answer in an instructive way. Our budget tool exists because of it.

```
# Streamable only if ROUTED. The 2 SHARED experts sit in the same namespace and  
# are NOT streamable, which is where hand arithmetic goes wrong.  
def classify(name: str) -> str:  
    if ".block_sparse_moe.experts." in name:  
        return "routed_expert"          # streamable: only 16 of 896 per token  
    if ".block_sparse_moe.shared_expert" in name:  
        return "shared_expert"          # RESIDENT: runs on every token  
    if ".self_attn." in name:  
        return "attention"              # resident  
    if "embed_tokens" in name or "lm_head" in name:  
        return "embedding"              # resident  
    return "other"                      # norms, router gates, biases: resident
```

The two shared experts run on every token, so they belong in the resident set even though their tensor names put them next to the routed experts. Getting that wrong makes the floor look smaller than it is, which is the worst direction to be wrong in.

What each preset actually costs in memory (Created by [Fareed Khan](#))

Those five budgets are the ones we will sweep at the end. For now the only thing that matters is the shape: the engine runs at 8 gigabytes and it runs at 224 gigabytes, and the difference is speed and nothing else.

One Small C Codebase, No BLAS, No Framework

The engine is six C files compiled into one binary. There is no BLAS, no PyTorch, no ONNX runtime, and no GPU library. The only dependencies are libm and OpenMP.

```
CFLAGS = -O3 -std=gnu99 -Wall -Wextra -Wpointer-arith -Wshadow -Wvla \
        -march=native -fopenmp -ffp-contract=off
LDFLAGS = -lm -fopenmp
```

The flag that looks unusual is `-ffp-contract=off`. By default a compiler is allowed to fuse a multiply and an add into a single FMA instruction, which changes the rounding. That is normally a good thing.

Here it is a problem, because we want the scalar path, the OpenMP path and the AVX2 path to produce **bit-identical** results, so that a performance change can never quietly become an accuracy change.

One C file plus small headers becomes a tiny static binary (Created by [Fareed Khan](#))

Before any of the components, the public header opens with a list of five invariants that must hold, each one a place where a plausible-looking implementation produces a model that runs, emits fluent text, and is wrong.

- **`A_log` is indexed per head, not per channel.** The checkpoint ships `head_dim` floats but only the first `num_heads` are meaningful, and the rest are padding.
- **The UT-transform inverse is $(I + A_{kk})^{-1}$.** The sign is not a convention.
- **`A_qk` retains its diagonal and `A_kk` does not.**
- **MLA uses NoPE, yet the 64 rope dimensions still exist and are still cached.** Only the rotation is absent, and dropping the slots changes the head width.

- **The MoE routing bias steers selection only.** The combining weights come from the unbiased sigmoid scores.

Read that list again, because it is the reason this post spends so much time on validation. Every one of those five is a mistake that compiles, runs, produces confident fluent English, and gives you a different model than the one you downloaded. There is no crash and no NaN to warn you.

I will tick each invariant off as we reach the component it belongs to.

The development machine here is Windows, which cannot build the engine at all because it needs `O_DIRECT`, `posix_memalign` and `getrusage`. So the portable subset gets compiled first, just to prove the code is clean, and then the whole thing is built on a small four core Linux box.

```
++ gcc -O2 -std=c99 -Wall -Wextra -Wno-unused-parameter test_tok.c -o test_tok.exe
++ gcc -O2 -std=c99 -Wall -Wextra -Wno-unused-parameter test_cfg.c k3_ops.c -o test_cfg.exe
```

```
148447 bytes  test_cfg.exe
150671 bytes  test_tok.exe
```

```
--- warnings: none (both compile clean under -Wall -Wextra) ---
```

```
OUTSTANDING: k3_run.c and k3_model.c have NOT been compiled or executed on Linux
```

That last line is the honest part. A clean compile of two files on Windows says nothing about the engine. So here is the full Linux build.

1. build, warnings are failures
-> clean build, no diagnostics


```
test_ops      97784 bytes
k3_model      89392 bytes
k3_run        179736 bytes
```

The whole inference engine is **179,736 bytes**. That is a 176 kilobyte binary whose job is to run a 1.56 terabyte model.

A 176 KB binary that runs a 1.56 TB model (Created by [Fareed Khan](#))

And here is the gate suite, which runs with no model weights at all.

```
22 passed, 0 failed, 0 skipped
```

```
ALL WEIGHTLESS GATES PASSED
```

```
This says the code is sound. It says NOTHING about the real 1.56 TB model.
```

I want that second line kept in view for the next several sections. Twenty-two passing kernel tests prove the arithmetic is right on fixtures. They prove nothing whatsoever about whether we are running Kimi K3.

One check does cross machines, though, and it is a good one. The tokenizer was run on the same input file under Windows and under Linux.

```
=== cross-platform tokenizer determinism ===  
Linux gcc 13.3.0 x86_64  
Windows gcc 16.1.0 x86_64  
input src/k3.h (24,499 bytes) -> 6,862 ids  
  
result IDENTICAL id streams  
  
(a naive md5 of stdout DIFFERS by one byte: Windows text-mode stdout writes the  
trailing newline as CRLF. That is the pipe, not the tokenizer.)
```

Two different compilers on two different operating systems produce the same 6,862 token ids from the same 24,499 bytes. The md5 sums differ by exactly one byte, and the reason is the line ending that the shell added, not anything the tokenizer did.

I like that this was chased down rather than waved away, because “the hashes differ” is exactly the kind of thing that gets ignored until it matters.

Reading a 1.56 TB Checkpoint From Its Headers

The checkpoint is 96 safetensors files. The format is deliberately simple, which is what makes it possible to treat 1.56 terabytes as an index rather than as data.

safetensors: one length, one header, then raw bytes at known offsets (Created by [Fareed Khan](#))

Every file starts with an 8 byte little-endian length, then that many bytes of JSON describing every tensor, then the raw tensor bytes back to back. The JSON gives each tensor a dtype, a shape and a byte range. Nothing is compressed and nothing is interleaved.

Index the shard, read the exact bytes on demand, then drop the pages (Created by [Fareed Khan](#))

We do not use a JSON library for this. The header can be tens of megabytes and we only want four fields per tensor, so the reader scans it directly.

```
/* Walk the header once, copy nothing we do not need. `p` sits just past the
 * opening quote of the tensor name. */
static const char *st_scan_entry(const char *p, const char *end, K3Tensor *t)
{
    const char *q = memchr(p, '"', (size_t)(end - p));
    if (!q || (size_t)(q - p) >= sizeof t->name) return NULL;
    memcpy(t->name, p, (size_t)(q - p));
    t->name[q - p] = '\0';
}
```

```

const char *d = st_find_key(q, end, "dtype");
if (!d) return NULL;
t->dtype = st_dtype_code(d);

const char *s = st_find_key(q, end, "shape");
if (!s) return NULL;
t->rank = 0;
t->nelem = 1;
for (const char *c = s; c < end && *c != ']'; c++) {
    if (*c >= '0' && *c <= '9') {
        long v = strtol(c, (char **)&c, 10);
        if (t->rank >= K3_ST_MAXRANK) return NULL;
        t->shape[t->rank++] = v;
        t->nelem *= (size_t)v;
    }
}

/* offsets are RELATIVE to the start of the data section */
const char *o = st_find_key(q, end, "data_offsets");
if (!o) return NULL;
t->off = (size_t)strtoull(o, (char **)&o, 10);
while (o < end && (*o < '0' || *o > '9')) o++;
t->nbytes = (size_t)strtoull(o, (char **)&o, 10) - t->off;
return o;
}

```

Every tensor found this way goes into a hash table keyed by a hash of its name, so a later lookup is a single probe rather than a walk. The choice of hash is not arbitrary.

```

/* Names are long and share deep prefixes
 * ("language_model.model.layers.N.block_sparse_moe.experts.M...."), so the hash
 * mix every byte; a prefix-only or length-only hash would pile every expert of
 * layer into one bucket. */
static uint64_t fnv1a(const char *s)
{
    uint64_t h = 1469598103934665603ull;
    while (*s) { h ^= (unsigned char)*s++; h *= 1099511628211ull; }
}

```

```
    return h;  
}
```

Half a million tensor names that all begin with the same forty characters is a genuinely hostile input for a hash function. FNV-1a mixes on every byte, so the expert index at the end of the name still moves the result.

Opening the model means reading 96 headers and building that table. It does not mean reading any weights.

Reading a tensor afterwards has one wrinkle. `O_DIRECT` requires the offset and the length to be multiples of the block size, and a tensor starts wherever the previous one ended.

```
int64_t k3_st_read_aligned(const K3St *s, int shard, int64_t off, int64_t nbytes  
                          void *buf, int64_t bufcap, int64_t *payload_off)  
{  
    /* widen outward to the enclosing aligned window */  
    const int64_t lo = off & ~(int64_t)(K3_ST_ALIGN - 1);  
    const int64_t hi = (off + nbytes + K3_ST_ALIGN - 1) & ~(int64_t)(K3_ST_ALIG  
    const int64_t len = hi - lo;  
    const int64_t pad = off - lo;  
    if (len > bufcap) return 0;  
    if (payload_off) *payload_off = pad;  
  
    int64_t got = 0;  
    while (got < len) {  
        ssize_t r = pread(dfd, (char *)buf + got, (size_t)(len - got), (off_t)(l  
        if (r <= 0) break; /* the last window may run past EOF */  
        got += r;  
    }  
    return got >= pad + nbytes ? nbytes : (got > pad ? got - pad : 0);  
}
```

The call reads a slightly larger window than asked for and returns the offset of the payload inside it, so the caller skips `pad` bytes and ignores the tail.

Note the `break` rather than a failure on a short read: the final aligned window of a shard extends past the end of the file, which is expected, so the return value checks that the payload itself was covered rather than that the whole window was.

This is also why the trunk gets its own file later. Once `pack_trunk.py` has aligned every layer, this widening disappears and the read becomes a plain `pread`.

Here is what that costs at full scale, taken from a generation run.

```
indexed 497220 tensors from 96 shards in 0.27 s
```

Half a million tensors indexed in about a quarter of a second. Five separate runs agreed on the tensor count and put the index time between 0.30 and 0.74 seconds, and this is the thing that makes everything afterwards possible: the engine never reads a shard it does not need, so the 1.56 terabytes on disk is a catalogue, not a working set.

A parser that agrees with itself proves nothing, so the index gets dumped and re-parsed independently in Python, comparing `dtype`, `shape`, `offsets`, and then the actual widened float bit patterns.

```
# Bit patterns, not tolerances: widening bf16 to f32 is lossless.
c_bits = np.asarray(c_values[name], dtype=np.float32).view(np.uint32)
p_bits = ref.astype(np.float32).view(np.uint32)

if not np.array_equal(c_bits, p_bits):
    bad = int(np.count_nonzero(c_bits != p_bits))
    fail(f"{name}: {bad} of {c_bits.size} float32 bit patterns differ")
```

And separately, before any of this is trusted, the download itself is verified against the published byte total.

```
=== shard verification ===
shards: 96
bytes: 1560936091448
expected: 1560936091448
RESULT: EXACT MATCH
```

The per-shard sizes are checked individually and not just the total, for two reasons. It turns a failure from “re-download 1.56 terabytes” into “re-download this one 17 gigabyte file”, and it catches the one case a total cannot: two shards wrong in opposite directions by the same amount.

The Config Reader That Refuses to Guess

Now we need the model’s dimensions. They come from the checkpoint’s own `config.json`, and this is the first place invariant number four can silently bite.

One-based layer indices, and 92 and 93 are both MLA by design (Created by [Fareed Khan](#))

Kimi K3 alternates two different attention mechanisms. Most layers use one, and every fourth layer uses the other, except that the last two layers are both the second kind so that the final layer always does global attention. The config lists those layers explicitly, and the list is **one-based**.

Here is what our reader prints when it reads the released config.

```
--- every value below is READ from the checkpoint, not assumed ---  
config: config.json (nested shape) | hidden=7168 layers=93 vocab=163840  
      | 24 MLA + 69 KDA | experts 896 top16 shared2 | latent=3584
```



```
--- KDA/MLA layer map (ONE-based, from full_attn_layers) ---
```

```
full_attn_layers (24, all MLA): 4,8,12,16,20,24,28,32,36,40,44,48,52,56,60,64,68
    note 92 AND 93 are both MLA - the report (2.1) places an extra Gated MLA layer
    at the end of the backbone so the final layer always does global attention.
kda_layers (69): every other layer.
```

Every one of those numbers is read from the file. None of them is compiled in. They land in one struct, which is worth showing whole because it is the entire model on one screen.

```
typedef struct {
    int hidden;           /* 7168 */
    int n_layers;         /* 93 */
    int vocab;            /* 163840 */
    float rms_eps;        /* 1e-5 */

    /* Kimi Delta Attention. 69 of the 93 layers. */
    int kda_heads;        /* 96 */
    int kda_head_dim;     /* 128, and d_k == d_v */
    int conv_k;           /* 4, depthwise, causal, SiLU fused */
    float gate_lb;        /* -5.0, the decay lower bound */

    /* Gated MLA. 24 of the 93 layers. */
    int n_heads;          /* 96 */
    int q_lora;           /* 1536 */
    int kv_lora;          /* 512 */
    int qk_nope;          /* 128 */
    int qk_rope;          /* 64, PRESENT BUT NEVER ROTATED */
    int v_head;           /* 128 */
    int mla_out_gate;     /* 1 */

    /* Stable LatentMoE. 92 of the 93 layers. */
    int n_experts;        /* 896 */
    int topk;             /* 16 */
    int n_shared;         /* 2, full width, added UNWEIGHTED */
    int latent;           /* 3584, the routed-expert width */
    int moe_inter;        /* 3072 */
    float routed_scale;   /* 1.0 */
    int moe_renorm;       /* 1 */
    int latent_norm;      /* 1, RMSNorm on the AGGREGATE, not per expert */
}
```

```
/* the single dense layer, layer 0 */
int first_dense;      /* 1 */
int dense_inter;      /* 33792 */

int attn_res_block;   /* 12. Boundaries fire when layer_idx % this == 0. */
float situ_b1;        /* 4.0 */
float situ_b2;        /* 25.0 */

int n_full_attn;      /* 24 */
int *full_attn;       /* ONE-BASED layer indices */
} K3Cfg;
```

That struct is the contract between the checkpoint and every kernel in the engine. If it is right, the model is Kimi K3. If any field is wrong, the model is something else that still speaks English, which is why the reader that fills it refuses to guess.

Refuse rather than guess, because a guessed field gives you a different model (Created by [Fareed Khan](#))

Consider what a permissive reader would do with this file. The released config nests its fields one level deeper than a fixture does, so a reader that only knows the flat shape finds nothing it recognises. If it then fills in defaults, two things happen.

The SiTU betas get 4.0 and 25.0, which are the correct values, so nothing looks wrong. And `full_attn_layers` comes back empty, so every one of the 93 layers runs as KDA and the 24 global-attention layers simply vanish. The

model loads, streams, decodes, and produces grammatical English from an architecture that is not Kimi K3.

That is the failure mode the five invariants warn about, and it is why the reader has exactly one rule: an absent field is an error, never a default.

```
/* An absent field is an ERROR, never a default. Missing names are accumulated s
 * the message lists all of them at once. */
static int cfg_req_int(jval root, const char *key, int *out,
                      const char **missing, int *nmissing)
{
    jval v = json_get(root, key);
    if (v.type != JSON_NUM) { /* absent OR the wrong type */
        if (*nmissing < K3_CFG_MAXMISS) missing[(*nmissing)++] = key;
        return 0;
    }
    *out = (int)v.num;
    return 1;
}
```

And it validates what it does find, because a present field can still be nonsense.

```
[no_layermap]
k3_cfg: no_layermap.json is missing 1 required field(s):
    full_attn_layers
refusing to substitute defaults: a config this reader cannot
fully understand would silently produce a DIFFERENT model.
ok    correctly rejected no_layermap.json

[bad_layer_index]
k3_cfg: bad_layer_index.json full_attn_layers[2] = 999 is outside 1..93
(the list is ONE-based)
ok    correctly rejected bad_layer_index.json
```

Two negative tests, both refusing to start. A config reader is about a hundred and fifty lines of the most boring code in the project, and it is one of exactly two places that can hand you a different model without telling you.

The Tokenizer, Byte for Byte

The other one is the tokenizer. Kimi K3 uses a byte-level BPE with 163,584 ranks plus 256 special tokens, shipped as a `tiktoken.model` file.

Every case goes through a file, never through argv (Created by [Fareed Khan](#))

The loader reads that file straight into the vendored BPE structures. The loading itself is mechanical, but it rests on three assumptions that each produce a tokenizer working perfectly on ASCII and diverging on everything else.

- **The merge keys are bytes, not code points.** A key that happens to decode as valid UTF-8 must still be treated as its raw bytes.
- **Ranks come from the file.** They are not derived from frequency at load time.
- **The added-token block is appended after the ranks,** so an added token's id is 163,584 plus its index, not its position in a merged table.

The test compares our C tokenizer against the Python `tiktoken` library case by case, and it does so through files rather than command line arguments, which sounds like a detail and is not.

```
oracle    : tiktoken 0.13.0
method    : token-for-token comparison; every case passed through a FILE, never a
            (argv is re-encoded to the active code page on Windows and would comp
            different bytes on every non-ASCII case)

PASS han only          2 ids
PASS japanese          6 ids
PASS korean            5 ids
PASS cyrillic          4 ids
PASS arabic            7 ids
PASS emoji zwj         5 ids
PASS code python       11 ids
PASS json              19 ids

tokenizer parity: 45/45 cases match
```

Forty-five cases, all matching, covering CJK, emoji with zero-width joiners, accented Latin, and whitespace runs. Then whole files are pushed through and decoded back.

```
roundtrip: 48353 bytes -> 14797 ids -> 48353 bytes : PASS    <- k3_ops.c
roundtrip: 24499 bytes -> 6862 ids -> 24499 bytes : PASS    <- k3.h
roundtrip: 201775 bytes -> 52671 ids -> 201775 bytes : PASS   <- REPORT.md
roundtrip: 53444 bytes -> 12145 ids -> 53444 bytes : PASS    <- modeling_kimi_k3.
```

Four files in, byte-identical files back out (Created by [Fareed Khan](#))

```

/* Greedily merge the lowest-rank adjacent pair. Everything here is BYTES. */
static int tok_encode_piece(const Tok *t, const unsigned char *p, int n, int *out)
{
    int parts[K3_TOK_MAXPIECE + 1], np = n + 1;
    for (int i = 0; i <= n; i++) parts[i] = i;          /* byte boundaries */

    for (;;) {
        int best = -1, bestrank = INT_MAX;
        for (int i = 0; i + 2 < np; i++) {
            const int r = tok_rank(t, p + parts[i], parts[i + 2] - parts[i]);
            if (r >= 0 && r < bestrank) { bestrank = r; best = i; }
        }
        if (best < 0) break;                             /* no mergeable pair left */
        memmove(&parts[best + 1], &parts[best + 2],
                (size_t)(np - best - 2) * sizeof(int));
        np--;
    }

    for (int i = 0; i + 1 < np; i++)
        out[i] = tok_rank(t, p + parts[i], parts[i + 1] - parts[i]);
}

```

```
    return np - 1;  
}
```

Two hundred kilobytes of markdown becomes 52,671 token ids and comes back as exactly the same two hundred kilobytes. That is the property we need, because every later claim about identical output rests on the tokenizer being deterministic.

The encoder itself is the standard byte-pair merge loop, and the only interesting part is that it works on bytes rather than characters throughout.

The loop keeps a list of slice boundaries and repeatedly joins whichever adjacent pair has the lowest rank in the merge table, which is what makes the result independent of any tie-breaking order. When no adjacent pair appears in the table at all, the piece is finished.



Reduction One: the Experts Already Ship at Half a Byte

Here is the first of the four reductions, and it is the largest single one.

The routed experts do not ship at bfloat16. They ship in **MXFP4**, which is a microscaling 4-bit float format. Each weight is a 4-bit nibble that indexes a 16-entry table, and every group of 32 consecutive weights shares one 8-bit exponent.

MXFP4: a 4-bit nibble scaled by one 8-bit exponent per 32 weights (Created by [Fareed Khan](#))

One byte carries two weights, and the low nibble is the even one (Created by [Fareed Khan](#))

That gives four bits per weight plus one byte per 32 weights for the shared scale.

Half a byte per weight plus the shared scale gives one expert exactly (Created by [Fareed Khan](#))

Half a byte plus one thirty-second of a byte is 0.53125 bytes per weight, and one expert has 33,030,144 parameters, so one expert is exactly 17,547,264 bytes. That matches the census from earlier to the byte, which is a good sign that we understand the format.

We can check the format against the released checkpoint rather than against the documentation. Our capture tool pulled the actual bytes of one expert's first weight matrix and recorded them alongside the values they should decode to.

```
{
  "note": "REAL Kimi K3 MXFP4 bytes from the released checkpoint. w = E2M1[nibbl",
  "source": "language_model.model.layers.1.block_sparse_moe.experts.0.w1",
  "rows": 64, "packed_cols": 1792, "scale_cols": 112,
  "logical_width": 3584, "group_size": 32,
  "e2m1_lut": [0.0, 0.5, 1.0, 1.5, 2.0, 3.0, 4.0, 6.0,
               -0.0, -0.5, -1.0, -1.5, -2.0, -3.0, -4.0, -6.0]
}
```

Sixty-four rows of 1,792 packed bytes decode to 64 rows of 3,584 weights, which is exactly two weights per byte. The 16 values in the lookup table are the entire vocabulary of an MXFP4 weight: zero, plus or minus a half, one, one and a half, two, three, four and six, scaled by a power of two.

Both halves of that decode are lookup tables, and building them is the only setup the format needs.

```

/* E2M1: sign, two exponent bits, one mantissa bit. Sixteen values in total. */
static const float K3_E2M1[16] = {
    0.0f, 0.5f, 1.0f, 1.5f, 2.0f, 3.0f, 4.0f, 6.0f,
    -0.0f, -0.5f, -1.0f, -1.5f, -2.0f, -3.0f, -4.0f, -6.0f
};

/* Byte -> its two weights, so the inner loop does one lookup, not two shifts. */
static void k3_pair_init(void)
{
    for (int b = 0; b < 256; b++) {
        K3_E2M1_PAIR[b][0] = K3_E2M1[b & 0x0F]; /* low nibble = EVEN element
        K3_E2M1_PAIR[b][1] = K3_E2M1[b >> 4]; /* high nibble = ODD element
    }
}

/* Scale byte -> power of two. 255 is NaN by spec, mapped to 0 to contain damage
static void k3_e8m0_init(void)
{
    for (int b = 0; b < 256; b++)
        K3_E8M0[b] = (b == 255) ? 0.0f : ldexpf(1.0f, b - 127);
}

```

Two tables of 256 entries is the entire cost of the format. Everything after this is ordinary arithmetic on floats that were never materialised anywhere.

Now, the nibble order. Each byte holds two weights, and which one comes first is a convention that cannot be checked by any statistic.

The low nibble is the EVEN element, and reversing it is silently wrong (Created by [Fareed Khan](#))

Every mean, every standard deviation, every histogram of the swapped version is identical to the correct one, because it is the same multiset of numbers. Only the positions differ. A verification that checks distributions would pass a matrix with every adjacent pair of weights transposed, and the model would be quietly wrong.

Now for the decision that saves the most time. The obvious way to use these weights is to decode them into floats and then do a normal matrix multiply. Let us price that.

The capture file carries a second array showing what you get if you read them the other way round, and its note says exactly why that array exists.

What dequantizing would cost, which is why we multiply from the nibbles (Created by [Fareed Khan](#))

```
"expected_swapped_nibbles": {  
  "note": "what you get if the low nibble is treated as the ODD element.  
Statistics are identical; positions are wrong."  
}
```

One expert at 17.55 megabytes becomes 132 megabytes once expanded to float32.

Each token touches 16 experts across 92 layers, which is 1,472 experts, so decoding them all would mean writing out **194 gigabytes per token** of pure format conversion, before a single multiply-accumulate.

Half a byte per weight saves 4 TB, and never dequantizing saves 194 GB a token (Created by [Fareed Khan](#))

So we never dequantize. The matrix multiply reads packed nibbles directly.

```
/* y[rows] = x[in] . W[rows][in], W stored as MXFP4. Nothing is dequantized. */  
void k3_matmul_mxfp4(float *y, const float *x, const unsigned char *packed,  
                    const unsigned char *scales, int in, int rows, int group)  
{
```

```

const int ngroup = (in + group - 1) / group;
const int rowbytes = (in + 1) / 2;          /* two nibbles per byte */

#pragma omp parallel for schedule(static)
for (int o = 0; o < rows; o++) {
    const unsigned char *pb = packed + (size_t)o * rowbytes;
    const unsigned char *sb = scales + (size_t)o * ngroup;
    double acc = 0.0;                      /* double, always */

    for (int g = 0; g < ngroup; g++) {
        const float s = K3_E8M0[sb[g]];
        if (s == 0.0f) continue;          /* a NaN scale zeroes the g

    const int i0 = g * group;
    const int n = (i0 + group <= in) ? group : (in - i0);
    float wf[64];

    /* low nibble is the EVEN weight, high nibble the ODD one */
    const int half = n / 2;
    for (int k = 0; k < half; k++) {
        const unsigned char b = pb[(i0 / 2) + k];
        wf[2 * k] = K3_E2M1_PAIR[b][0]; /* low nibble -> even inde
        wf[2 * k + 1] = K3_E2M1_PAIR[b][1]; /* high nibble -> odd index
    }
    if (n & 1) wf[n - 1] = K3_E2M1_PAIR[pb[(i0 / 2) + half]][0];

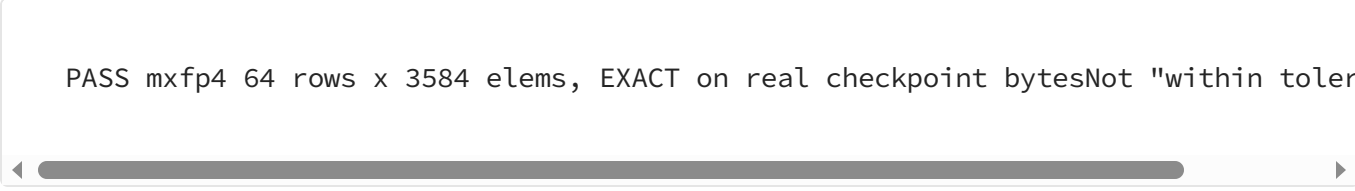
    double part = 0.0;
    for (int k = 0; k < n; k++) part += (double)x[i0 + k] * (double)wf[k]
    acc += part * (double)s;
}
y[o] = (float)acc;
}
}

```

Two details in there are worth pausing on. The `if (s == 0.0f) continue` line handles a scale byte of 255, which the MXFP4 specification defines as NaN. We map it to zero so that one corrupt byte kills one group of 32 weights instead of turning an entire row into NaN and then poisoning every downstream layer.

And the line `if (n & 1)` handles a group with an odd number of elements. With a group size of 32 that can never happen on this checkpoint, and it is written anyway. That is the difference between code that works and code that is correct, and it costs one line.

Here is the gate that checks all of this against the checkpoint bytes.



```
PASS mxfp4 64 rows x 3584 elems, EXACT on real checkpoint bytesNot "within toler
```

Exact. Both sides are reading identical bytes off identical weights, so there is nothing here that could legitimately differ, and the test is written with zero tolerance to say so.

That is reduction one. The experts arrive at 0.53125 bytes per weight instead of 2, which takes 5.45 terabytes of expert weights down to 1.447 terabytes, and never expanding them saves another 194 gigabytes of memory traffic per token.

Kernels With a Floating Point Contract

Before building the layers, we need the small numeric pieces they are made of, and we need one rule about how they are allowed to compute.

The rule is that every code path must produce the same bits. RMSNorm is the most used kernel in the model.

RMSNorm with epsilon inside the square root, accumulated in double (Created by Fareed Khan)

```
void k3_rmsnorm(float *out, const float *x, const float *w, int n, float eps)
{
    double ss = 0.0; /* double, not float */
    for (int i = 0; i < n; i++) ss += (double)x[i] * (double)x[i];
    const float inv = (float)(1.0 / sqrt(ss / (double)n + (double)eps));
    for (int i = 0; i < n; i++) out[i] = x[i] * inv * (w ? w[i] : 1.0f);
}
```

Two details there are load-bearing. The accumulator is a **double** even though every input and output is a float, and epsilon goes **inside** the square root rather than outside.

The dot product needs the same bit-stability while also being fast, so the reduction order is fixed by construction.

A fixed reduction order, so scalar and AVX2 agree bit for bit (Created by [Fareed Khan](#))

```
/* Four accumulators partitioned by i % 4. This pins the summation order, so a
 * 4-wide vector loop adds the same numbers in the same sequence. */
static float k3_dot4(const float *x, const float *w, int n)
{
    double a0 = 0.0, a1 = 0.0, a2 = 0.0, a3 = 0.0;
    int i = 0;
    for (; i + 3 < n; i += 4) {
        a0 += (double)x[i] * (double)w[i];
        a1 += (double)x[i + 1] * (double)w[i + 1];
        a2 += (double)x[i + 2] * (double)w[i + 2];
        a3 += (double)x[i + 3] * (double)w[i + 3];
    }
    for (; i < n; i++) a0 += (double)x[i] * (double)w[i];
    return (float)((a0 + a1) + (a2 + a3)); /* the parentheses are the cont
}
```

The trunk weights are bfloat16, and widening them is not a conversion at all.

bf16 to fp32 is a shift, not a conversion, so widening is lossless (Created by [Fareed Khan](#))

A bfloat16 value is simply the top 16 bits of a float32 with the bottom 16 bits dropped. So widening is a shift left by 16 into the high half, and it is exact. That is why the trunk can be streamed in its shipped precision without any accuracy question at all, a point that becomes important much later.

That shift is also what makes the vectorised trunk matmul possible without a conversion table, and it is worth seeing what “bit-identical” costs in practice.

```
void k3_matmul_bf16(float *y, const float *x, const uint16_t *W, int in, int out)
{
    #pragma omp parallel for schedule(static) if (out > 64)
        for (int o = 0; o < out; o++) {
            const uint16_t *row = W + (size_t)o * in;
            int i = 0;
            double acc;
            #if defined(__AVX2__)
            {
                __m256d v = _mm256_setzero_pd();
                for (; i + 3 < in; i += 4) {
                    /* bf16 -> f32 is a 16-bit shift: no table, no rounding */
                    const __m128i h = _mm_loadl_epi64((const __m128i *) (row + i));
                    const __m128i b32 = _mm_slli_epi32(_mm_cvtepu16_epi32(h), 16);
                    const __m256d wd = _mm256_cvtps_pd(_mm_castsi128_ps(b32));
                    const __m256d xd = _mm256_cvtps_pd(_mm_loadu_ps(x + i));
                    v = _mm256_add_pd(v, _mm256_mul_pd(wd, xd)); /* NOT fmadd */
                }
                double a[4];
                _mm256_storeu_pd(a, v);
            }
        }
}
```

```

        acc = (a[0] + a[1]) + (a[2] + a[3]);
    }
    #else
    {
        double a0 = 0.0, a1 = 0.0, a2 = 0.0, a3 = 0.0;
        for (; i + 3 < in; i += 4) {
            a0 += (double)k3_bf16f(row[i]) * (double)x[i];
            a1 += (double)k3_bf16f(row[i + 1]) * (double)x[i + 1];
            a2 += (double)k3_bf16f(row[i + 2]) * (double)x[i + 2];
            a3 += (double)k3_bf16f(row[i + 3]) * (double)x[i + 3];
        }
        acc = (a0 + a1) + (a2 + a3);
    }
    #endif
    for (; i < in; i++) acc += (double)k3_bf16f(row[i]) * (double)x[i];
    y[o] = (float)acc;
}
}

```

Look at the two branches side by side. Both accumulate into four doubles, both partition by $i \% 4$, and both reduce as $(a0 + a1) + (a2 + a3)$. The vector path is the scalar path with the same additions performed in the same order, four at a time.

And note the comment on the multiply-add. `_mm256_add_pd` of an `_mm256_mul_pd` is deliberately not `_mm256_fmadd_pd`.

A fused multiply-add rounds once instead of twice and is therefore **more** accurate, which is exactly the problem: it would give a different answer from the scalar loop, and here a different answer is worse than a slightly less accurate one, because a hardware capability must never change the output.

Same weights, three code paths, one hash (Created by [Fareed Khan](#))

Now, how do we actually prove that the AVX2 path and the scalar path agree? A tolerance check will not do it, because a tolerance check happily passes a kernel that quietly reassociated its sum and drifted in the last bits.

So the benchmark hashes the output instead: it runs FNV-1a over the exact bit pattern of every float in the result, and the test is to build the file twice, once with AVX2 and once without, and diff the two hex digests. Either they are the same number or they are not.

Here is the fixture suite running the whole set of kernels against reference values generated from PyTorch.

```
tolerance: atol=1.0e-05 rtol=1.0e-04 (from MANIFEST.json)
PASS  rmsnorm          n=384    worst=0.01x tol
PASS  situ_glu         n=48     worst=0.00x tol
      bound check |out|=100.000 must be <= b1*b2=100.0 : ok
PASS  kda_decay        H=4 D=16 tok=4 max|dg|=4.768e-07 max|dalp|alpha|=1.788e-07
PASS  mla              n=768    worst=0.08x tol
      H=4 qh=32 (nope 24 + rope 8) v=16 kv_lora=32 scale=0.176777
PASS  mxfp4            64 rows x 3584 elems, EXACT on real checkpoint bytes
```

```
PASS  matmul_bf16  n=129  bit-identical to k3_matmul  
22 passed, 0 failed, 0 skipped
```

The worst case across all 22 kernels is 8 percent of the allowed tolerance, and two of them are exact rather than merely close. The `matmul_bf16` line is the one I care about most: the bfloat16 path is **bit-identical** to the float32 path, which is what lets us keep the trunk in its shipped precision without changing a single output bit.

Where one token goes on the floor configuration: 80% of it is waiting on disk (Created by [Fareed Khan](#))

One last thing before the layers. When the kernels are benchmarked at the model's own dimensions on the smallest configuration, the split is about 36 seconds of trunk reading, 11 seconds of expert reading and 10 seconds of actual arithmetic.

Eighty percent of a token is waiting for a disk. Keep that in mind through the next few sections, because it is the reason the second half of this post is about I/O and not about kernels.

Reduction Two: KDA, Attention With a Memory That Never Grows

Sixty-nine of the 93 layers use Kimi Delta Attention, and its property that matters for fitting the model is easy to state: its memory does not grow with context length.

A standard attention layer stores a key and a value for every token it has seen, so its cache grows linearly forever. KDA instead keeps one fixed-size matrix per head and updates it in place as tokens arrive.

The recurrent state is the same size at 10 tokens and at 100,000 (Created by [Fareed Khan](#))

Ninety-six heads times a 128 by 128 matrix per head is the entire memory of a KDA layer, at any sequence length. Across all 93 layers that is **626.25 megabytes**, and it is 626.25 megabytes whether you feed it ten tokens or a million.

Every token folds into the same fixed-size state (Created by [Fareed Khan](#))

Why 69 layers are KDA: its state does not grow with context (Created by [Fareed Khan](#))

That plot is the argument for the whole design. The green line is flat forever. The red line is what the other 24 layers cost, and it crosses our machine's memory somewhere around 100,000 tokens.

If all 93 layers behaved like the red line, the model would not fit at any context length worth having.

So how does the layer work? Let us walk it in order.

Decay the state, read from it, write the delta, then read the updated state (Created by [Fareed Khan](#))

First, the projections go through a short depthwise causal convolution of width four with the SiLU activation fused into it.

A depthwise causal convolution of width 4 with the activation fused in (Created by [Fareed Khan](#))

```

/* Causal depthwise conv with SiLU fused. State is carried across calls. */
void k3_shortconv(float *y, const float *x, const float *w, float *state,
                  int channels, int k, int T)
{
    const int hist = k - 1; /* guard on hist, not on buf */
    float *buf = hist ? (float *)malloc((size_t)hist * sizeof(float)) : NULL;
    if (hist && !buf) k3_fatal_oom("ShortConv history", (size_t)hist * sizeof(float));

    for (int c = 0; c < channels; c++) {
        if (hist) {
            if (state) memcpy(buf, state + (size_t)c * hist, (size_t)hist * sizeof(float));
            else      memset(buf, 0, (size_t)hist * sizeof(float));
        }

        for (int t = 0; t < T; t++) {
            const float cur = x[(size_t)t * channels + c];
            float acc = w[(size_t)c * k + hist] * cur; /* taps run oldest to newest */
            for (int j = 0; j < hist; j++)
                acc += w[(size_t)c * k + j] * buf[j];

            for (int j = 0; j + 1 < hist; j++) buf[j] = buf[j + 1];
            if (hist > 0) buf[hist - 1] = cur;

            y[(size_t)t * channels + c] = acc * sigmoidf_(acc); /* SiLU, fused */
        }
        if (state && hist) memcpy(state + (size_t)c * hist, buf, (size_t)hist * sizeof(float));
        free(buf);
    }
}

```

The history lives in the same carried state as the recurrent matrix and is updated in place, so a streaming decode never re-reads anything.

Note the guard on `hist` rather than on `buf`: with a kernel width of one there is no history at all, `malloc(0)` is allowed to return `NULL`, and a check on the pointer would silently skip the entire convolution and leave the output untouched.

Then the queries and keys are L2-normalised. Note that this is a sum of squares and not a mean of squares, which is a different function and looks nearly identical in code.

A sum of squares, not a mean, and applied to q and k only (Created by Fareed Khan)

Then comes the decay gate, and this is **invariant number one**.

The decay gate, with A indexed per head and not per channel (Created by Fareed Khan)

```
void k3_kda_decay(float *g, float *alpha, const float *z, const float *A_log,
                  const float *dt_bias, int H, int D, float lb)
{
    for (int h = 0; h < H; h++) {
        const float a = expf(A_log[h]);          /* PER HEAD, not per channel */
        for (int d = 0; d < D; d++) {
```

```

const int i = h * D + d;
const float u = a * (z[i] + dt_bias[i]);
const float gi = lb * sigmoidf_(u); /* in (lb, 0] */
g[i] = gi;
alpha[i] = expf(gi); /* in (e^lb, 1] */
    }
}
}

```

The gate lower bound is minus five, so `alpha` lands somewhere in the interval from e^{-5} to 1. A value near one means this key channel keeps almost all of its history, and a value near e^{-5} means it forgets almost everything, per channel and per token.

Now the recurrence itself. The order of these four steps is the part that is easy to get subtly wrong.

The delta rule: decay, read, write the difference, then read again (Created by [Fareed Khan](#))

```

void k3_kda_step(float *S, float *o, const float *q, const float *k,
                const float *v, const float *alpha, float beta, int dk, int dv)
{
    /* 1. decay: scale ROW i of S by alpha[i], per key channel */
    for (int i = 0; i < dk; i++) {
        float *row = S + (size_t)i * dv;

```

```

    const float a = alpha[i];
    for (int j = 0; j < dv; j++) row[j] *= a;
}

/* 2. read the state along k:  $u = S^T k$  */
float *u = (float *)calloc((size_t)dv, sizeof(float));
if (!u) k3_fatal_oom("KDA recurrence temporary", (size_t)dv * sizeof(float))
for (int i = 0; i < dk; i++) {
    const float ki = k[i];
    if (ki == 0.0f) continue;
    const float *row = S + (size_t)i * dv;
    for (int j = 0; j < dv; j++) u[j] += ki * row[j];
}

/* 3. rank-one delta write:  $(v - u)$  is the prediction error */
for (int i = 0; i < dk; i++) {
    const float ki = k[i];
    if (ki == 0.0f) continue;
    float *row = S + (size_t)i * dv;
    for (int j = 0; j < dv; j++) row[j] += ki * beta * (v[j] - u[j]);
}

/* 4. output from the ALREADY UPDATED state:  $o = S^T q$  */
for (int j = 0; j < dv; j++) o[j] = 0.0f;
for (int i = 0; i < dk; i++) {
    const float qi = q[i];
    if (qi == 0.0f) continue;
    const float *row = S + (size_t)i * dv;
    for (int j = 0; j < dv; j++) o[j] += qi * row[j];
}
free(u);
}

```

The comment on step two is worth reading twice. That `calloc` happens **after** step one has already scaled the state, so an early return on allocation failure would leave the recurrent matrix permanently decayed but never updated. Every subsequent token would then be computed from a state that is quietly wrong, with nothing to indicate it.

That is why the failure path aborts instead of returning.

Now let us assemble those pieces into a layer, because the order they run in is the architecture.

Nine ordered steps, and the numbering is not decoration (Created by [Fareed Khan](#))

```
void k3_kda_layer(float *out, const float *x, const K3KdaW *w, const K3Cfg *c,
                 int T, float *state, float *scratch)
{
    const int E = c->hidden, H = c->kda_heads, D = c->kda_head_dim;
    const int P = H * D, K = c->conv_k, hist = K - 1;

    float *q = scratch;
    float *k = q + (size_t)T * P;
    float *v = k + (size_t)T * P;
    float *z = v + (size_t)T * P;
    float *a1 = z + (size_t)T * P;
    float *bt = a1 + (size_t)T * P;
    float *o = bt + (size_t)T * P;
    float *gb = o + (size_t)T * P;
    float *wr = gb + P;
    float *fa = wr + P;

    /* 1. projections */
    for (int t = 0; t < T; t++) {
        const float *xt = x + (size_t)t * E;
        k3_mmw(q + (size_t)t * P, xt, w->q, w->wdt, E, P);
        k3_mmw(k + (size_t)t * P, xt, w->k, w->wdt, E, P);
        k3_mmw(v + (size_t)t * P, xt, w->v, w->wdt, E, P);
        k3_mmw(bt + (size_t)t * H, xt, w->b, w->wdt, E, H);
        k3_mmw(fa, xt, w->f_a, w->wdt, E, D); /* one low-rank pair, all h
        k3_mmw(z + (size_t)t * P, fa, w->f_b, w->wdt, D, P);
    }

    /* 2. ShortConv with fused SiLU, carrying state across calls */
    float *cs = state ? state + (size_t)H * D * D : NULL;
    k3_shortconv(q, q, w->q_conv, cs ? cs : NULL, P, K, T);
    k3_shortconv(k, k, w->k_conv, cs ? cs + (size_t)P * hist : NULL, P, K, T);
    k3_shortconv(v, v, w->v_conv, cs ? cs + (size_t)2 * P * hist : NULL, P, K, T);

    /* 3. L2Norm on q and k ONLY, per head. v is deliberately left alone. */
    for (int t = 0; t < T; t++)
```

```

    for (int h = 0; h < H; h++) {
        l2norm_(q + (size_t)t * P + (size_t)h * D, D, 1e-6f);
        l2norm_(k + (size_t)t * P + (size_t)h * D, D, 1e-6f);
    }

    /* 4/5. beta and the decay chain */
    for (int t = 0; t < T; t++) {
        for (int h = 0; h < H; h++) bt[(size_t)t * H + h] = sigmoidf_(bt[(size_t)t * H + h],
            k3_kda_decay(z + (size_t)t * P, a1 + (size_t)t * P, z + (size_t)t * P,
                w->A_log, w->dt_bias, H, D, c->gate_lb);
    }

    /* 6. recurrence, per head, with q pre-scaled by d_k^-0.5 */
    float *S = state;
    float *Sown = NULL;
    if (!S) {
        Sown = (float *)calloc((size_t)H * D * D, sizeof(float));
        if (!Sown) k3_fatal_oom("KDA recurrent state", (size_t)H * D * D * sizeof(float));
        S = Sown;
    }
    const float qscale = 1.0f / sqrtf((float)D);
    for (int t = 0; t < T; t++)
        for (int h = 0; h < H; h++) {
            const size_t off = (size_t)t * P + (size_t)h * D;
            for (int i = 0; i < D; i++) wr[i] = q[off + i] * qscale;
            k3_kda_step(S + (size_t)h * D * D, o + off, wr, k + off, v + off,
                a1 + off, bt[(size_t)t * H + h], D, D);
        }

    /* 7/8/9. head-wise RMSNorm, THEN the gate, THEN the output projection */
    for (int t = 0; t < T; t++) {
        const float *xt = x + (size_t)t * E;
        float *ot = o + (size_t)t * P;
        for (int h = 0; h < H; h++)
            k3_rmsnorm(ot + (size_t)h * D, ot + (size_t)h * D, w->o_norm, D, c->gate_lb);
        k3_mmw(gb, xt, w->g, w->wdt, E, P);
        for (int i = 0; i < P; i++) ot[i] *= sigmoidf_(gb[i]);
        k3_mmw(out + (size_t)t * E, ot, w->o, w->wdt, P, E);
    }
    free(Sown);
}

```

Nine numbered steps, and the numbering is not decoration. Step 3 normalises q and k and deliberately leaves v alone. Step 6 pre-scales the

query by one over the square root of the head dimension before the recurrence rather than after it.

And steps 7, 8 and 9 are the tail that `verify_kda.py` exists to pin down.

Norm first, then gate, then project, and that order is not interchangeable (Created by [Fareed Khan](#))

That ordering is where **invariants two and three** live, and it is also the single most valuable check in the entire project. The released model ends the layer with a fused kernel from the `fla` library called `FusedRMSNormGated`, which takes the raw gate and applies the sigmoid internally.

Our reference instead does an explicit RMSNorm followed by a multiply by the sigmoid of the gate. Those are the same function **only** if the fused kernel is norm first and gate second, so `tools/verify_kda.py` proves it rather than assuming it, and its reason is worth quoting:

The plausible alternatives, gating before norming or norming the gate, both run

That sentence should be on the wall of anyone reimplementing a released architecture. No test catches it except comparing against the released code, which is exactly what that script does.

Let us see the layer run at full width. This test allocates one full-width 96-head, 128-dimension KDA layer and pushes tokens through it, with no checkpoint involved.

cleanly and produce a different model.

```
one KDA layer   : 443740384 params  (887.48 MB at bf16)
69 KDA layers  : 61.24 GB at bf16   (KDA attention ONLY, not the whole trunk)
full trunk     : 113.49 GB at bf16, 56,743,648,000 always-active params
one expert     : 33030144 params  (17.55 MB at MXFP4)
all 82432 experts: 1.45 TB at MXFP4  <- streamed from NVMe

per-sequence state, FIXED regardless of context:
  KDA recurrent : 217.06 MB at bf16
  ShortConv     : 15.26 MB at bf16
  MLA KV        : 2.37 MB per position (24 MLA layers, EXPANDED k and v, fp32)
                  19.38 GB at 8192 context
                  310.04 GB at 131072 context

allocating and running ONE real-width KDA layer (fp32)...
weights: 1.77 GB
ran 4 tokens in 0.17 s (44 ms/token)
output all finite: YES, max |y| = 0.000003
state non-zero   : yes
```

There is a lot in that block. One KDA layer is 887 megabytes of weights, and all 69 of them together are 61.24 gigabytes, which is more than half the entire resident trunk.

Four tokens run through a full-width layer in 0.17 seconds with finite output and a state that actually changed, which tells us the recurrence is wired up and not silently producing zeros.

And the two lines under “FIXED regardless of context” are the payoff. The KDA state is 217 megabytes and the convolution history is 15 megabytes, and neither number moves no matter how long the sequence gets. Compare that to the line below them, where the other attention mechanism costs 310 gigabytes at 131,072 positions.

That is reduction two. Sixty-nine of 93 layers have a memory cost that is completely independent of how much text you feed them.

Reduction Three: MLA, One Latent Instead of Ninety-Six Heads

The other 24 layers do global attention, because a purely recurrent stack cannot look back at an arbitrary earlier token with full precision. Those layers use Gated Multi-head Latent Attention, and they are the ones that cost memory per position, so we want them to cost as little as possible.

A normal attention layer with 96 heads would store, for every position, a key and a value for each head. MLA instead projects the token down into one small shared latent, caches only that, and rebuilds the per-head keys and values from it when they are needed.

One latent per position is cached, and k and v are rebuilt on use (Created by [Fareed Khan](#))

The query and the token both pass through a small shared latent (Created by [Fareed Khan](#))

The latent is 512 dimensions for the key and value content plus 64 more, and those 64 are where **invariant number four** lives.

```
/* NoPE: the 64 rope dimensions are projected and cached, but never rotated. */
static void k3_mla_project(float *q, float *kv, const float *x, const K3MlaW *w,
                          const K3Cfg *c)
{
    float qa[K3_MAX_QLORA];                /* down to 1536, norm, back up */
    k3_mmw(qa, x, &w->q_a_proj, c->q_lora, c->hidden);
    k3_rmsnorm(qa, qa, w->q_a_norm, c->q_lora, c->rms_eps);
    k3_mmw(q, qa, &w->q_b_proj, c->n_heads * (c->qk_nope + c->qk_rope), c->q_lora);

    /* one 576-wide projection: the entire per-position cache for this layer */
    k3_mmw(kv, x, &w->kv_a_proj, c->kv_lora + c->qk_rope, c->hidden);
    k3_rmsnorm(kv, kv, w->kv_a_norm, c->kv_lora, c->rms_eps);
    /* the trailing qk_rope floats are left unnormalised and unrotated */
}
```

So the head width is 192, which is 128 content dimensions plus those 64 carried-but-unrotated ones, and the softmax scale is the inverse square root of 192 rather than of 128. Getting that wrong is a change of about 22 percent in every attention score, and it produces perfectly readable output.

Here is the scoring loop, which is where those two pieces meet.

```
for (int t = 0; t < T; t++) {
    const int p = cached + t;
    for (int h = 0; h < H; h++) {
        const float *qt = q + ((size_t)t * H + h) * qh;
        float m = -INFINITY;
        for (int s = 0; s <= p; s++) {
            /* causal: s <= p */
            const float *ks = K3_KV_AT(s) + (size_t)h * kvd;
            const float *kr = K3_ROPE_AT(s); /* shared slot */
            double d = 0.0;
            for (int i = 0; i < qn; i++) d += (double)qt[i] * (double)ks[i];
            /* the rope slot is UNROTATED but still scored, and the SAME 64
             * values serve every head */
            for (int i = 0; i < qr; i++) d += (double)qt[qn + i] * (double)kr[i];
            sc[s] = (float)d * scale;
            if (sc[s] > m) m = sc[s];
        }
        double z = 0.0;
        for (int s = 0; s <= p; s++) { sc[s] = expf(sc[s] - m); z += sc[s];

```

The score is two dot products added together. The first runs over the 128 content dimensions, which are per head. The second runs over the 64 rope dimensions, which are **shared**: `K3_ROPE_AT(s)` takes no head index, so all 96 heads score against the same 64 numbers.

That is what makes the cache 576 wide instead of 96 times 320, and skipping the second term is the quiet way to get a model that still speaks.

The `K3_KV_AT` and `K3_ROPE_AT` macros are how one function serves both decode paths. When a cache is supplied they index into it, and when it is not they index into scratch, so incremental decode and full recompute run the same code with no branch in the inner loop.

Now, how much does a position cost? The engine caches expanded keys and values in float32 across all 24 MLA layers.

Twenty-four MLA layers, expanded k and v in What MLA caches per position, per layer (Created by Fareed Khan)

p32, per position (Created by Fareed Khan)

That is 2.37 megabytes per position, which is where the ladder in the previous section came from. It is worth seeing what the alternative would have been, because the released reference makes a different choice: its `KimiDynamicCache` stores the **expanded** keys and values, at 96 heads times 320 floats per position per layer.

The released code caches expanded heads, the latent is 53x smaller (Created by [Fareed Khan](#))

Ninety-six heads times 320 floats is 30,720 floats per position per layer. The latent is 576. That is **53 times smaller for mathematically identical output**, and it is the difference between a context length you can use and one you cannot.

There is a second reason `tools/verify_mla.py` exists, and it is the more important one. Our C engine is checked against a PyTorch reference that we wrote, and that reference is otherwise only checked against itself, so a shared misreading of the architecture would pass every test in the project.

That script compares our reference against the actual released model class, which closes the loop.

Because the KV cache is the one thing that grows, the engine refuses to start rather than discovering the problem an hour in.

```
/* The context limit is the MLA KV cache, not any array size. */
if (incremental) {
    const double kv_need = (double)(np + gen + 1) * K3_KV_BYTES_PER_POS;
    const double avail    = mem_available_bytes();
    if (avail > 0.0 && kv_need > avail * 0.9) {
        fprintf(stderr,
            "\nREFUSING: the KV cache for %d positions needs %s but only %s is\n"
            "available. This is a MEMORY limit, not an engine ceiling: MLA cache\n"
            "expanded k and v in fp32 across 24 layers, so context costs ~2.37 M\n"
            "position regardless of budget. Shorten the request, or use full\n"
            "recompute (drop --incremental), which carries no KV cache at all.\n"
            np + gen + 1, kb, ab);
        return 2;
    }
}
```

I like error messages that tell you which of your assumptions was wrong. “This is a MEMORY limit, not an engine ceiling” saves somebody an afternoon of looking for a hardcoded constant that does not exist.

That is reduction three. Context costs 2.37 megabytes per position instead of 125, and the number that grows is as small as it can be made.

Attention Residuals: Layers That Look Back

There is one more structural piece before the feed-forward path, and it is unusual enough to be worth a section even though it costs no memory.

In a normal transformer each layer adds its output to a running residual stream. Kimi K3 does something different: each layer attends over the

outputs of every preceding **block**, where a block is twelve layers, and learns how much of each to mix in.

Each layer attends over the outputs of every preceding block (Created by [Fareed Khan](#))

Blocks of twelve, so the residual stack never exceeds nine sources (Created by [Fareed Khan](#))

Every twelve layers the running prefix is snapshotted and cleared (Created by [Fareed Khan](#))

The implementation is a small softmax over a handful of source vectors.

```
void k3_attn_res(float *out, const float *src, const float *fold,
                int nsrc, int n, float eps)
{
    float *score = (float *)malloc((size_t)nsrc * sizeof(float));
    if (!score) k3_fatal_oom("AttnRes scores", (size_t)nsrc * sizeof(float));

    for (int s = 0; s < nsrc; s++) {
        const float *v = src + (size_t)s * n;
        double ss = 0.0;
        for (int i = 0; i < n; i++) ss += (double)v[i] * (double)v[i];
        const float inv = (float)(1.0 / sqrt(ss / (double)n + (double)eps));
        double acc = 0.0; /* key: the NORMALISED source vector */
        for (int i = 0; i < n; i++) acc += (double)(v[i] * inv) * (double)fold[i];
        score[s] = (float)acc;
    }

    float m = score[0];
    for (int s = 1; s < nsrc; s++) if (score[s] > m) m = score[s];
    double z = 0.0;
    for (int s = 0; s < nsrc; s++) { score[s] = expf(score[s] - m); z += score[s]; }

    for (int i = 0; i < n; i++) out[i] = 0.0f;
}
```

```

    for (int s = 0; s < nsrc; s++) {
        const float p = (float)(score[s] / z);
        const float *v = src + (size_t)s * n;    /* the RAW source, not the key */
        for (int i = 0; i < n; i++) out[i] += p * v[i];
    }
    free(score);
}

```

The keys are normalised before scoring and the values are the **raw** sources. Norming the values as well is the obvious-looking mistake, and it quietly rescales the residual stream. And `fold` arrives pre-multiplied, because the norm gain and the scoring projection collapse into one vector and there is no reason to do that per source.

We now have every piece of a layer, so here is the whole thing, which is also where the block boundary lives.

One layer: aggregate, attend, aggregate again, then route (Created by [Fareed Khan](#))

```

void k3_decoder_layer_inc(float *h, float *block_residual, int *n_blocks,
                        const K3LayerW *w, const K3Cfg *c, int layer_idx,
                        int T, float *state, float *scratch,
                        float *kvc, float *ropec, int cached, int cap)
{
    const int E = c->hidden;
    const int maxb = c->n_layers / c->attn_res_block + 2;

    float *pref    = scratch;                                /* [T][E] the running residual

```



```

float *tmp      = pref + (size_t)T * E;      /* [T][E] module output
float *hin      = tmp + (size_t)T * E;      /* [T][E] normalised layer input
float *foldA    = hin + (size_t)T * E;      /* [E] attention aggregator
float *foldM    = foldA + E;                /* [E] mlp aggregator
float *src      = foldM + E;                /* [maxb+1][E] source stack
float *dgu      = src + (size_t)(maxb) * E; /* [2*dense_inter]
float *sub      = dgu + (size_t)2 * c->dense_inter; /* scratch for the modul

/* norm gain and scoring projection collapse into one vector */
for (int i = 0; i < E; i++) {
    foldA[i] = w->attn_res_norm[i] * w->attn_res_proj[i];
    foldM[i] = w->mlp_res_norm[i] * w->mlp_res_proj[i];
}

memcpy(pref, h, (size_t)T * E * sizeof(float));
int have_prefix = 1; /* mirrors "prefix_sum is not No

/* aggregation before attention, only when snapshots already exist */
if (*n_blocks > 0) {
    for (int t = 0; t < T; t++) {
        for (int b = 0; b < *n_blocks; b++)
            memcpy(src + (size_t)b * E,
                block_residual + ((size_t)t * maxb + b) * E,
                (size_t)E * sizeof(float));
        memcpy(src + (size_t)(*n_blocks) * E, pref + (size_t)t * E,
            (size_t)E * sizeof(float));
        k3_attn_res(h + (size_t)t * E, src, foldA, *n_blocks + 1, E, c->rms_
    }
}

/* block boundary: snapshot the running residual, then CLEAR it */
if (layer_idx % c->attn_res_block == 0) {
    for (int t = 0; t < T; t++)
        memcpy(block_residual + ((size_t)t * maxb + *n_blocks) * E,
            pref + (size_t)t * E, (size_t)E * sizeof(float));
    (*n_blocks)++;
    have_prefix = 0;
}

/* whichever attention was bound for this layer */
for (int t = 0; t < T; t++)
    k3_rmsnorm(hin + (size_t)t * E, h + (size_t)t * E, w->in_norm, E, c->rms
if (w->kda) k3_kda_layer(tmp, hin, w->kda, c, T, state, sub);
else      k3_mla_cached(tmp, hin, w->mmla, c, T, sub, kvc, ropec, cached, c

if (have_prefix) for (size_t i = 0; i < (size_t)T * E; i++) pref[i] += tmp[i]
else              { memcpy(pref, tmp, (size_t)T * E * sizeof(float)); have_pr

/* aggregation before the MLP, with no emptiness guard */
for (int t = 0; t < T; t++) {

```

```

for (int b = 0; b < *n_blocks; b++)
    memcpy(src + (size_t)b * E,
           block_residual + ((size_t)t * maxb + b) * E,
           (size_t)E * sizeof(float));
memcpy(src + (size_t)(*n_blocks) * E, pref + (size_t)t * E,
       (size_t)E * sizeof(float));
k3_attn_res(h + (size_t)t * E, src, foldM, *n_blocks + 1, E, c->rms_eps)
}

for (int t = 0; t < T; t++)
    k3_rmsnorm(hin + (size_t)t * E, h + (size_t)t * E, w->post_norm, E, c->r

if (w->moe) {
    int idx[K3_MAX_TOPK]; float wt[K3_MAX_TOPK];
    k3_moe(tmp, hin, w->moe, c, T, idx, wt, sub);
}
}

```

The two aggregations are not symmetric: the one before attention is skipped when no snapshots exist yet, and the one before the MLP has no such guard, because the reference has none either. And `have_prefix` exists to distinguish “add to the running residual” from “this is the first layer of a new block, so replace it”.

Now, does that actually do anything observable? It does, and the reference forward pass draws it for us. When the PyTorch reference ran the full 93 layers, it printed the maximum absolute activation after every layer.

```

L45 KDA MoE 41.2 s |h| max 47.714829
L46 KDA MoE 38.9 s |h| max 62.183392
L47 MLA MoE 44.3 s |h| max 76.281532
L48 KDA MoE 36.1 s |h| max 2.902113
L49 KDA MoE 39.5 s |h| max 4.353188

```

Look at layer 47 and then layer 48. The acti

That is a block boundary: the running prefix was snapshotted and cleared, so the next layer starts from a fresh small residual.

Plot all 93 of those values and the architecture draws itself.

Attention residuals drawn by the data: activations climb, then collapse every 12 layers (Created by [Fareed Khan](#))

Seven sawteeth, one per block, each one climbing for twelve layers and then collapsing. I did not draw that shape, the model did. The biggest peak is 136.7 at layer 59, which drops to 1.78 at layer 60.

When a measured curve has exactly the period your code says it should, that is a decent sign the code matches the model.

Picking 16 Experts of 896

Now the feed-forward path, which is where all those experts finally get used.

The router scores every one of the 896 experts and picks sixteen. And this is **invariant number five**, which is the subtlest of the lot.

The bias steers selection only, the weights come from unbiased scores (Created by [Fareed Khan](#))

```

/* Score all n_experts, pick the top-k, weight them. The bias steers SELECTION o
void k3_router(int *idx, float *w, const float *x, const float *W, const float *
               int hidden, int n_experts, int topk, int renorm, float routed_sca
{
    float sc[K3_MAX_EXPERTS], ch[K3_MAX_EXPERTS];

#pragma omp parallel for schedule(static)
    for (int e = 0; e < n_experts; e++) {
        double acc = 0.0;
        for (int i = 0; i < hidden; i++)
            acc += (double)x[i] * (double)W[(size_t)e * hidden + i];
        sc[e] = 1.0f / (1.0f + expf(-(float)acc));      /* the UNBIASED score */
        ch[e] = sc[e] + (bias ? bias[e] : 0.0f);      /* the SELECTION score */
    }

    char taken[K3_MAX_EXPERTS] = {0};                /* top-k by repeated max */
    for (int j = 0; j < topk; j++) {
        int best = -1;
        for (int e = 0; e < n_experts; e++)
            if (!taken[e] && (best < 0 || ch[e] > ch[best])) best = e;
        taken[best] = 1;
        idx[j] = best;
        w[j] = sc[best];                             /* the UNBIASED score a
    }

    if (renorm) {
        float s = 0.0f;
        for (int j = 0; j < topk; j++) s += w[j];
    }

```

```
        if (s > 0.0f) for (int j = 0; j < topk; j++) w[j] /= s;
    }
    for (int j = 0; j < topk; j++) w[j] *= routed_scale;
}
```

Notice that `sc[e]` and `ch[e]` are both computed and they are used for different things. The biased score picks the winners and the unbiased score weights them. Collapsing those two into one variable is a two-character edit that changes the model.

The experts themselves do not run at the full 7,168-dimensional width. The token is projected down into a narrow latent first, the experts work there, and the result is projected back up.

Experts run in a narrow latent, and the norm is on the aggregate (Created by [Fareed Khan](#))



Route, run 16 experts in a 3584-wide latent, then project back up (Created by [Fareed Khan](#))

```

/* Stable LatentMoE for one token.
 *
 * Six steps, and step 4 is the one people get wrong: the RMSNorm is applied to
 * AGGREGATE of the weighted expert outputs, not to each expert individually. No
 * per expert and then summing is a different function.
 *
 * The two shared experts run on the ORIGINAL full-width input, not the latent,
 * their output is added UNWEIGHTED. They are not part of the top-k sum. */
void k3_moe(float *out, const float *x, const K3MoeW *w, const K3Cfg *c,
            int T, int *idx, float *wt, float *scratch)
{
    const int E = c->hidden, L = c->latent, I = c->moe_inter;
    float *z = scratch, *accL = z + L, *gu = accL + L, *act = gu + 2 * I;
    float *edn = act + I;

    for (int t = 0; t < T; t++) {
        const float *xt = x + (size_t)t * E;

        /* 1. route on the FULL width, not the latent */
        k3_router(idx, wt, xt, w->gate, w->gate_bias, E,
                  c->n_experts, c->topk, c->moe_renorm, c->routed_scale);

        /* 2. down-project into the expert latent */
        k3_mmw(z, xt, w->down, w->wdt, E, L);

        /* 3. run the chosen experts, accumulating in the latent */
        for (int i = 0; i < L; i++) accL[i] = 0.0f;
        if (w->src && w->src->getmany) w->src->getmany(w->src, w->layer, idx, c-

        for (int j = 0; j < c->topk; j++) {
            K3ExpertQ q;
            if (w->src->get(w->src, w->layer, idx[j], &q) != 0) {
                k3_expert_drops++;          /* counted, never silent */
                continue;
            }
            k3_matmul_mxfp4(gu,      z, q.p1, q.s1, L, I, K3_MXFP4_GROUP);
            k3_matmul_mxfp4(gu + I, z, q.p3, q.s3, L, I, K3_MXFP4_GROUP);
            k3_situ_glu(act, gu, I, c->situ_b1, c->situ_b2);
            k3_matmul_mxfp4(edn, act, q.p2, q.s2, I, L, K3_MXFP4_GROUP);
            for (int i = 0; i < L; i++) accL[i] += wt[j] * edn[i];
        }

        /* 4. norm the AGGREGATE, not each expert */
        if (c->latent_norm) k3_rmsnorm(accL, accL, w->latent_norm, L, c->rms_eps

```

```

/* 5. back up to full width */
float *ot = out + (size_t)t * E;
k3_mmw(ot, accL, w->up, w->wdt, L, E);

/* 6. shared experts, on the ORIGINAL input, added UNWEIGHTED */
const int SI = I * c->n_shared;
k3_mmw(gu,      xt, w->sh1, w->wdt, E, SI);
k3_mmw(gu + SI, xt, w->sh3, w->wdt, E, SI);
k3_situ_glu(act, gu, SI, c->situ_b1, c->situ_b2);
k3_mmw(edn, act, w->sh2, w->wdt, SI, E);
for (int i = 0; i < E; i++) ot[i] += edn[i];
}
}

```

The activation inside each expert is SiTU-GLU, which is a gated unit with both halves passed through bounded tanh functions.

SiTU-GLU with beta1 = 4 and beta2 = 25, so the product is bounded (Created by [Fareed Khan](#))

```

void k3_situ_glu(float *y, const float *x, int n, float b1, float b2)
{
    const float *gate = x;
    const float *up    = x + n;
    for (int i = 0; i < n; i++) {
        const float g = gate[i];
        /* the sigmoid takes the UNCAPPED gate */
        const float a = b1 * tanhf(g / b1) * sigmoidf_(g);
        const float u = b2 * tanhf(up[i] / b2);
        y[i] = a * u;
    }
}

```

```
}  
}
```

Look at where the sigmoid takes its argument. It reads the **uncapped** gate g , not the capped $b1 * \tanh(g / b1)$. Feeding it the capped value gives a function that is still smooth, still bounded, still produces fluent output, and is a different activation.

The comment cites the exact line of the released model file, which is the only way to settle a question like that.

Because both factors are bounded, the product can never exceed 4 times 25, which is 100. Our fixture drives it deliberately to that exact analytic cap and checks it.

```
PASS situ_glu      n=48      worst=0.00x tol  
      bound check |out|=100.000 must be <= b1*b2=100.0 : ok
```

The output reaches exactly 100.000 and does not exceed it. A fixture that only tested the near-linear region would pass an implementation with the caps left out entirely, because for small inputs the tanh is almost the identity function.

Now here is the thing to remember for later. The Kimi K3 technical report describes a training technique called Quantile Balancing whose entire purpose is to **flatten expert usage** across the pool, so that no small set of experts dominates.

The hottest experts, out of 10,010 distinct ones touched (Created by [Fareed Khan](#))

That is good for the model and it is going to be very bad for our cache. Hold that thought.

One last thing about that `continue` in the expert loop, because it connects to how the whole program ends. A dropped expert means one token was computed with a fifteen-sixteenths of its routed sum, and the run still finishes and still prints a plausible token. So the engine counts them globally and refuses to exit successfully.

```
/* Silent numerical corruption that exits 0 is indistinguishable from a good run
if (k3_expert_drops) {
    fprintf(stderr,
        "\nRUN INVALID: %ld routed expert load(s) failed and were dropped fr
        "the MoE sum. The token ids above are CORRUPT. Re-run; if this repea
        "the shard set or the storage is at fault.\n", k3_expert_drops);
    return 4;
}
return 0;
```

Note where that sits: after all the reporting and all the frees. A corrupt run still prints its full diagnostics and still cleans up. It simply does not exit zero.

That is reduction three completed, or rather it is the reason reduction three works. Because only 16 of 896 experts run per layer, the 1.45 terabytes of expert weights never has to be resident. It only has to be reachable.

Packing the Trunk: 93 Layers, One Read Each

We have got the resident set down to 113.49 gigabytes. That is still far more than a consumer machine has, and it is the last big number in the ledger.

The trunk is spread across 96 shard files, interleaved with the experts. Reading one layer's worth of it means finding a few dozen tensors scattered through a 17 gigabyte file. So before running anything, we rewrite it once into a layout that suits how we actually read it.

Refuse if the bytes are not contiguous, because a gap means copying experts (Created by [Fareed Khan](#))

The key fact that makes this cheap is that each layer's trunk tensors already sit in one contiguous run inside its shard. So packing is 93 range copies, not a scatter-gather.

```
# 93 sequential range copies, one per layer, each verified contiguous first.
for L in range(n_layers):
    names = [n for n in index if n.startswith(f"model.layers.{L}.") and not is_e
    shards = {index[n] for n in names}
    if len(shards) != 1:
        die(f"layer {L} spans {len(shards)} shards; refusing")

    runs = sorted((offsets[n][0], offsets[n][1]) for n in names)
    lo, hi = runs[0][0], runs[-1][1]
    covered = sum(b - a for a, b in runs)
    if covered != hi - lo:
        # a gap would drag expert bytes along with it
        die(f"layer {L} is not contiguous: {hi - lo - covered} bytes of gap")

    out_off = align_up(out_off, ALIGN)      # head aligned for O_DIRECT
    copy_range(shard_path(shards.pop()), lo, hi, out_fh, CHUNK)
    out_off += align_up(hi - lo, ALIGN)     # and the tail, so reads never overr
```

O_DIRECT needs both ends aligned, which costs 4 KB per layer (Created by [Fareed Khan](#))

Note the refusal in the middle. If a layer's tensors were not contiguous, copying the whole span would drag expert bytes along with it and bloat the trunk file. Rather than silently produce a 400 gigabyte trunk, the packer stops.

Here is the packer running on the released checkpoint.

```
packed 10/93 layers, 12.90 GB, 25 s (521 MB/s)
packed 20/93 layers, 24.31 GB, 47 s (520 MB/s)
packed 30/93 layers, 36.14 GB, 74 s (488 MB/s)
packed 40/93 layers, 47.55 GB, 99 s (480 MB/s)
packed 50/93 layers, 59.38 GB, 127 s (466 MB/s)
packed 60/93 layers, 70.79 GB, 153 s (462 MB/s)
packed 70/93 layers, 82.62 GB, 181 s (457 MB/s)
packed 80/93 layers, 94.02 GB, 208 s (451 MB/s)
packed 90/93 layers, 105.86 GB, 236 s (448 MB/s)
packed 93/93 layers, 108.81 GB, 244 s (447 MB/s)
```

```
wrote /models/k3trunk/trunk.bin: 108.81 GB across 93 layers
largest layer run: 2.341 GB <- the streaming slot size
```

Packing 93 contiguous layer runs into one 108.81 GB file (Created by [Fareed Khan](#))

Four minutes, once, and we get a 108.81 gigabyte file where layer L lives at a known offset and can be read in a single call. The last line is the one that sizes everything downstream: **the largest layer run is 2.341 gigabytes**, so any buffer that has to hold one arbitrary layer must be at least that big.

Reduction Four: Streaming the Trunk Turns a Floor Into a Dial

Here is the last reduction, and it is the one that decides whether this runs on your machine.

The trunk is 108.81 gigabytes and every single layer of it is used on every single token. There is no sparsity to exploit and nothing to skip. So the question is not how to avoid reading it, it is where to keep it.

This is the point where the whole memory hierarchy finally exists, so here it is in one picture.

The path one token takes, from cold NVMe to the next word (Created by [Fareed Khan](#))

Pinned layers cost nothing to read, everything else comes through one ring slot (Created by [Fareed Khan](#))

The design is a pinned prefix plus one rotating slot. Whatever fits in the budget gets pinned permanently, and the rest cycles through a single ring buffer. Critically, it is a **prefix** and not a cache, and the reason is worth stating carefully.

The engine walks layers 0 through 92 in the same order on every token. That is a cyclic scan, and a cyclic scan is the pathological case for least-recently-used eviction: by the time layer 0 comes round again it is the least recently used thing in the cache, so it has always just been evicted.

An LRU of 90 slots over a 93 layer cycle achieves a hit rate of **exactly zero**. Pinning the first N layers instead gives a deterministic hit rate of N over 93, which for N of 90 is 96.8 percent.

The obvious data structure is not merely suboptimal here, it is wrong in the worst possible direction, returning zero where the trivial approach returns almost one.

Sizing the pinned set and the ring slot is mutually dependent, since the slot has to be big enough for the largest unpinned layer, so the code iterates to a fixed point.

```
/* Pinned count and slot size are mutually dependent, so iterate to a fixed point */
size_t slot = tr->max_run; /* start assuming nothing is pinned */
int npin = 0;
for (int pass = 0; pass < 4; pass++) {
    size_t avail = budget > slot * (size_t)nring ? budget - slot * (size_t)nring : budget;
    int n = 0;
    size_t used = 0;
    while (n < tr->n_layers && used + tr->lay[n].nbytes <= avail) {
        used += tr->lay[n].nbytes;
        n++;
    }
    size_t need = 0; /* largest run still not pinned */
    for (int L = n; L < tr->n_layers; L++)
        if (tr->lay[L].nbytes > need) need = tr->lay[L].nbytes;
    if (need == 0) need = 4096; /* everything pinned: degenerate but */
    if (n == npin && need == slot) break; /* converged */
    npin = n;
    slot = k3_align_up(need, K3_TRUNK_ALIGN);
}
```

There is a memory detail that turns out to matter a lot. `O_DIRECT` reads pin their destination pages, and a 2.37 gigabyte slot on 4 kilobyte pages is about

578,000 pages, pinned and unpinned 93 times per token. That is nearly 54 million page operations per token purely in bookkeeping, so the arenas are allocated on 2 megabyte hugepages instead.

Once a layer's bytes land in a slot they still have to become kernel arguments, and the binder splits the tensors into two classes to do it. Big matrices stay in the checkpoint's bfloat16 and are simply tagged, so the matmul dispatches to the bf16 path that we already proved is bit-identical.

Small elementwise vectors, the norm weights and biases and the router gate, are widened to float32 once at bind time, because a kernel that reads them element by element would otherwise pay for that shift on every single access.

Before allocating any of this, the engine adds up everything it is about to need and compares it against what the kernel says is available.

Everything is summed before anything is allocated, then compared to free RAM (Created by [Fareed Khan](#))

```
/* Add up EVERYTHING before allocating anything. */
const double need_b = w_trunk + w_model + w_cache + w_state + w_buf + w_kv;
const double have = mem_available_bytes();          /* MemAvailable, not MemFree */

if (need_b > have * 0.95) {
    fprintf(stderr,
        "\nREFUSING TO START: this needs %s and the machine has %s "
```



```

        "available, a shortfall of %s.\n"
        "Options: a larger box, a smaller --cache-gb, or fewer --layers.\n",
        b6, b1, b2);
    return 1;
}

```

That plan is explicitly a forecast and not a result. It omits the safetensors index, which is about 78 megabytes at full scale, it reports requested budgets rather than actual reservations, and it cannot see fragmentation. So the engine also measures the peak resident set afterwards and labels it, in the output, as the number to quote.

We now have every piece, so here is the whole forward pass. This is one token, end to end, and it is where the streaming actually happens.

One token: embed, walk 93 layers, aggregate, project to the vocabulary (Created by [Fareed Khan](#))

```

/* One full forward over T tokens, writing logits for the LAST position only. */
static int forward(Weights *w, const K3Cfg *c, K3Cache *cache, const int *ids, i
        float *logits_last, float *scratch, float *h, float *br, floa
{
    const int E = c->hidden;
    const int maxb = c->n_layers / c->attn_res_block + 2;
    const int P = c->kda_heads * c->kda_head_dim;
    const size_t kper = (size_t)P * c->kda_head_dim + (size_t)3 * P * (c->conv_k

    for (int t = 0; t < T; t++)
        k3_embed_row(h + (size_t)t * E, w->mb.embed, w->mb.wdt, ids[t], E);

    memset(br, 0, (size_t)T * maxb * E * sizeof(float));
    /* incremental decode carries the KDA state forward; full recompute rebuilds
    if (!w->kvc) memset(kstate, 0, kper * (size_t)w->n_bound * sizeof(float));

```

```

int nb = 0;
for (int L = 0; L < w->n_bound; L++) {
    /* bring this layer in, and hint the next one so its read overlaps */
    if (w->trunk) {
        if (k3_trunk_bind(w->trunk, c, L, &w->lay[L]) != 0) {
            fprintf(stderr, "trunk bind failed at layer %d\n", L);
            return -1;
        }
        k3_trunk_prefetch(w->trunk, L + 1);
    }
    if (w->lay[L].lay.moe) {
        w->lay[L].moe.src = &cache->src;
        w->lay[L].moe.layer = L;
    }
    if (w->kvc && w->m1a_slot[L] >= 0) {
        const size_t kvper = (size_t)w->kv_cap * c->n_heads * (c->qk_nope +
        const size_t rpper = (size_t)w->kv_cap * c->qk_rope;
        const int mi = w->m1a_slot[L];
        k3_decoder_layer_inc(h, br, &nb, &w->lay[L].lay, c, L, T,
            kstate + kper * (size_t)L, scratch,
            w->kvc + kvper * (size_t)mi,
            w->ropec + rpper * (size_t)mi,
            w->cached, w->kv_cap);
    } else {
        k3_decoder_layer_inc(h, br, &nb, &w->lay[L].lay, c, L, T,
            kstate + kper * (size_t)L, scratch,
            NULL, NULL, 0, 0);
    }
}

/* one model-level aggregator, beyond the two in every layer */
if (w->mb.out_res_norm && w->mb.out_res_proj) {
    float *fold = scratch;
    float *src = fold + E;
    for (int i = 0; i < E; i++) fold[i] = w->mb.out_res_norm[i] * w->mb.out_
    for (int t = 0; t < T; t++) {
        for (int b = 0; b < nb; b++)
            memcpy(src + (size_t)b * E, br + ((size_t)t * maxb + b) * E,
                (size_t)E * sizeof(float));
        memcpy(src + (size_t)nb * E, h + (size_t)t * E, (size_t)E * sizeof(f
        k3_attn_res(h + (size_t)t * E, src, fold, nb + 1, E, c->rms_eps);
    }
}

float *nrm = scratch;
k3_rmsnorm(nrm, h + (size_t)(T - 1) * E, w->mb.norm, E, c->rms_eps);
k3_mmw(logits_last, nrm, w->mb.lm_head, w->mb.wdt, E, c->vocab);
return 0;
}

```

That is the entire model in seventy lines. Embed the tokens, walk 93 layers binding each one as it arrives, apply one final aggregator across all the block snapshots, normalise the last position and project it to 163,840 logits.

Two details carry a lot of weight. The `if (!w->kvc)` line is the whole distinction between incremental decode and full recompute in a single condition: incremental carries the KDA state and the convolution history forward, full recompute rebuilds them from scratch every step.

And the prefetch hint on the next line is unconditionally correct, because the walk order is fixed at 0 through 92 on every token. The engine always knows the layer it will need next before it needs it, which is a luxury a cache never has.

A fixed walk order means the next read can start before this layer finishes (Created by [Fareed Khan](#))

Let us actually run it. This is the first time the engine touched all 2.78 trillion parameters.

5. REAL MODEL, 2.78T params, 96 shards from the volume

Kimi K3, pure C, real checkpoint

shards : /models/k3

prompt : 5 tokens, generating 2

indexed 497220 tensors from 96 shards in 0.70 s

memory plan

trunk (STREAMED) 16.00 GB

embed + lm_head 4.70 GB

expert cache 6.00 GB

recurrent state 626.25 MB

buffers 6.68 MB

KV cache 0.00 B

TOTAL 27.33 GB

available 65.91 GB

trunk stream: 108.81 GB packed, 10/93 layers PINNED (13.16 GB), ring 1 x 2.37 GB
reads use O_DIRECT (page cache bypassed)
deterministic hit rate 10.8% (a cyclic scan defeats LRU, so a pinned prefix is used instead)

peak RSS after loading weights: 4.78 GB (the plan above is a forecast, this is
expert cache: 341 slots x 17.56 MB = 5.99 GB (0.41% of the 1.45 TB expert pool)

STEP	TOKEN	SECONDS	CACHE HIT	READ GB	TOK/S
0	2494	167.84	35.0	83.91	0.006
1	9	171.65	39.3	94.05	0.006

2 tokens in 339.5 s, 169.75 s/token average

PEAK RSS for the whole run: 25.83 GB <- quote this, not the plan

It is also **169.75 seconds per token**, which is unusable. Almost everything in the second half of this post is about walking that number down to 10.66, and the interesting part is that the thing that fixes it is not the thing I expected.

Now let us push the other way and see how small this goes. Here is a run with nothing pinned at all and an expert cache of under two gigabytes.

```
memory plan
  trunk (STREAMED) 4.00 GB
  embed + lm_head 4.70 GB
  expert cache     2.00 GB
  recurrent state 626.25 MB
  buffers          9.36 MB
  TOTAL           11.33 GB

trunk stream: 108.81 GB packed, 0/93 layers PINNED (0.00 GB), ring 2 x 2.37 GB
               deterministic hit rate 0.0%
expert cache: 113 slots x 17.56 MB = 1.98 GB (0.14% of the 1.45 TB expert pool)
```

STEP	TOKEN	SECONDS	CACHE HIT	READ GB	TOK/S
0	17374	57.08	22.8	99.70	0.018
1	20829	27.72	0.0	25.83	0.036
2	10	26.95	0.0	25.83	0.037
3	427	27.25	0.0	25.83	0.037

```
cache [final step]
  requests      : 1472  hits 0 (0.00%)  misses 1472  evictions 1472
```

An expert cache holding **0.14 percent** of the expert pool. A cache hit rate of exactly **zero**, with all 1,472 requests missing and all 1,472 evicting.

And it still emits 17374, 20829, 10, 427, which is the same correct answer every other configuration gives.

Streaming turns a 315 GB floor into an 11 GB dial (Created by [Fareed Khan](#))

Compare that to what holding the trunk resident costs. That path spends **150 seconds just loading weights** before the first token, asks for 315 gigabytes, and needs a machine with 755 gigabytes free to be allowed to start.

That is reduction four, and it closes the ledger. Streaming the trunk turns the last 113 gigabyte floor into a number you choose.

One last piece of the streaming path is the read itself, and it is short because the packing did the hard work.

```
/* Offset and length are both 4096-aligned, so this is a plain pread with no fix
static int load_run(K3Trunk *tr, int L, unsigned char *dst)
{
    const K3Run *r = &tr->lay[L];
    size_t got = 0;
    while (got < r->nbytes) {
        const ssize_t n = pread(tr->fd, dst + got, r->nbytes - got,
                                (off_t)(r->off + got));
        if (n <= 0) return -1;      /* a short read is a corrupt layer */
    }
}
```

```
    got += (size_t)n;
}
tr->bytes_read += got;
return 0;
}
```

Ninety-three of those per token, in a fixed order, is the whole trunk. At the top preset ninety of them never happen because those layers are already pinned, which is where the throughput comes from.

An LRU Cache for the Experts

The trunk is handled. Now the other side of the read path: 1,472 expert fetches per token, each one 17.56 megabytes, drawn from a 1.45 terabyte pool.

The cache is an arena of fixed-size slots, one expert per slot, with three states.

A slot is empty, reserved but not yet readable, or holding an expert (Created by [Fareed Khan](#))

The expert cache holds whole experts, so the budget divides exactly (Created by [Fareed Khan](#))

The middle state is what makes a parallel batch safe, and the victim selector is where all three states meet.

```
/* Three slot states, not two: a key, EMPTY, or INFLIGHT. */
static int pick_victim(K3Cache *c)
{
    int best = -1;
    uint64_t oldest = (uint64_t)-1;
    for (int i = 0; i < c->nslot; i++) {
        if (c->key_of[i] == K3_SLOT_INFLIGHT) continue; /* being read into RIG
        if (c->key_of[i] == K3_SLOT_EMPTY) return i; /* free, take it */
        if (c->pinned[i]) continue;
        if (c->used_at[i] < oldest) { oldest = c->used_at[i]; best = i; }
    }
    return best;
}
```

Three details in twelve lines. `INFLIGHT` slots are skipped entirely rather than treated as candidates, so a slot cannot be claimed twice. `EMPTY` returns immediately rather than joining the scan, because a free slot is always a better choice than evicting a live one.

And pinned slots are skipped after the empty test rather than before it, so pinning never blocks the cheap path.

The eviction itself is plain least-recently-used over a monotonic counter, which is deliberate. There is no frequency term and no scan resistance, because the routing distribution turns out not to reward one, for reasons the next section measures.

The other reason the cache is interesting is how it issues reads.

Reserve serially, read in parallel, then publish only what arrived (Created by [Fareed Khan](#))

Batched preads keep the device busy, serial gets leave it idle (Created by [Fareed Khan](#))

```
static int cache_getmany(K3ExpertSrc *self, int layer, const int *experts, int n
{
    K3Cache *c = (K3Cache *)self->ctx;
    int slots[K3_MAX_TOPK];
```

```

/* phase 1: reserve serially, so no two experts take the same slot */
int nres = 0;
for (int j = 0; j < n; j++) {
    int s = cache_lookup(c, layer, experts[j]);
    if (s >= 0) { slots[j] = -1; continue; } /* already resident */
    s = cache_pick_victim(c);
    if (s < 0) { slots[j] = -1; continue; }
    c->slot_id[s] = K3_SLOT_INFLIGHT;
    slots[j] = s;
    nres++;
}

/* phase 2: read in parallel, in disk-offset order */
int order[K3_MAX_TOPK];
cache_sort_by_offset(c, layer, experts, slots, n, order);

#pragma omp parallel for schedule(dynamic)
for (int k = 0; k < n; k++) {
    const int j = order[k];
    if (slots[j] < 0) continue;
    if (!k3_expert_load_direct(c->st, layer, experts[j], c->arena + slot_off
        slots[j] = -2;
}

/* phase 3: publish only what arrived */
for (int j = 0; j < n; j++) {
    if (slots[j] >= 0) c->slot_id[j] = expert_key(layer, experts[j]);
    else if (slots[j] == -2) c->slot_id[j] = K3_SLOT_EMPTY;
}
return nres;
}

```

The slot sizing carries a similar guard. An expert is 17,547,264 bytes, which happens to be exactly 4,284 times 4,096, so the alignment the reads need holds on the released checkpoint **by coincidence**.

Code that assumed the alignment rather than enforcing it would therefore work on every shipped weight, which is why the cache fixture deliberately uses a non-conforming expert size. The released weights cannot exercise that path, so the test data was built to.

One more piece, because it is where the 17.55 megabytes actually come off the disk. An expert is six tensors, three packed weight matrices and three scale arrays, and in the released shards they sit back to back. So the loader checks for that and, when it holds, fabricates a fake tensor covering the whole run.

```
int64_t k3_expert_load(const K3St *, const K3ExpertRef *r, unsigned char *buf)
{
    if (r->contiguous) {                                     /* one coalesced 17.55 MB read */
        K3Tensor t;
        memset(&t, 0, sizeof t);
        t.name = (char *)"expert";
        t.shard = r->shard;
        t.off = r->off;
        t.nbytes = r->nbytes;
        t.dtype = K3_DT_U8;
        t.ndim = 1;
        t.shape[0] = r->nbytes;
        return k3_st_read(s, &t, buf);
    }

    /* fallback: six separate reads, one per tensor */
    static const char *W[3] = { "w1", "w2", "w3" };
    char name[256];
    int64_t got = 0;
    for (int i = 0; i < 3; i++) {
        snprintf(name, sizeof name, EXPERT_FMT, r->layer, r->expert, W[i], "weig
        const K3Tensor *p = k3_st_find(s, name);
        snprintf(name, sizeof name, EXPERT_FMT, r->layer, r->expert, W[i], "weig
        const K3Tensor *c = k3_st_find(s, name);
        if (!p || !c) return got;
        got += k3_st_read(s, p, buf + (p->off - r->off));
        got += k3_st_read(s, c, buf + (c->off - r->off));
    }
    return got;
}
```

That synthetic `K3Tensor` named "expert" does not correspond to anything in the checkpoint. It exists purely so the six-tensor run can be handed to the

same short-read loop that reads any other tensor, which lets a special case reuse the general read path.

Here are the cache gates.

```
4. streaming expert cache
  PASS  prefetch_reads <= hits           requests 24, hits 24, prefetch 24
  PASS  mixed batch and serial           0 of 24 wrong
CACHE TESTS PASSED
```

And one measurement that shapes expectations. The expert reader is timed under three access patterns: sequential cold, which is the best case and a fiction, random cold, and warm.

An eightfold spread in storage speed, and the engine is I/O bound (Created by [Fareed Khan](#))

Random cold is the one that matters, because the router picks experts by relevance and the file lays them out by index, and those two orders have nothing to do with each other.

How Big Should That Cache Be? Ask the Trace

Before spending memory on the expert cache, it would be good to know how much is worth spending. Measuring that directly would mean running the model once per cache size on a machine large enough to hold the whole expert pool, which is not a thing we have.

There is a shortcut, and it is a good one. **Routing does not depend on the cache.** The same prompt picks the same experts in the same order no matter what the cache does, so one run can record every (layer, expert) request and that single trace can then be replayed at any capacity under any policy.

One run, one trace, then replay it at every capacity (Created by [Fareed Khan](#))

We replay it under three. LRU is what the engine runs. Belady evicts whatever is needed furthest in the future, which requires the whole trace in advance and is exactly why no online cache can implement it, so it stands as a ceiling.

And a pinned hot set plus LRU sits between them.

```
def belady(trace, cap):
    """Evict whatever is needed furthest in the future. A ceiling, not a policy.
    nxt = defaultdict(deque)
    for i, k in enumerate(trace):
        nxt[k].append(i)
    resident, hits = set(), 0
    for k in trace:
        nxt[k].popleft()
        if k in resident:
            hits += 1
            continue
        if len(resident) >= cap:
            victim = max(resident, key=lambda r: nxt[r][0] if nxt[r] else 1 << 6
            resident.discard(victim)
        resident.add(k)
    return hits / len(trace)
```

Here is the whole curve, from one run.

trace: 100096 requests, 10010 distinct experts, about 68 token(s)
 distinct experts touched: 10010 of 82432 (12.14% of the pool)
 if nothing were cached: 25.83 GB per token
 total reuse: 90086 of 100096 requests are repeats (90.0%)

CACHE	SLOTS	LRU	BELADY	PIN+LRU	GB READ/TOK	SEC/TOK
8 GB	455	36.24%	39.42%	37.82%	16.47	13.35
16 GB	911	36.24%	42.61%	39.42%	16.47	13.35
32 GB	1823	36.24%	48.99%	42.57%	16.47	13.35
64 GB	3647	36.24%	61.74%	48.66%	16.47	13.35
128 GB	7294	49.19%	84.59%	62.86%	13.12	10.64
192 GB	10941	90.00%	90.00%	90.00%	2.58	2.09
1450 GB	82633	90.00%	90.00%	90.00%	2.58	2.09

compulsory misses: 10010 (every expert must be read at least once), a ceiling of 90.00% hit rate for ANY policy at ANY size on this trace.

Every distinct expert must be read once, so no policy can beat this (Created by [Fareed Khan](#))

Note the working set at the top of that block. Sixty-eight tokens touched 10,010 distinct experts, which is only 12.14 percent of the pool.

The experts this trace touched at all, against a 1.45 TB pool (Created by [Fareed Khan](#))

Holding every expert this trace touched would need 175.65 gigabytes, against 1,446 gigabytes for the whole pool. So even a perfect cache for this workload is a fraction of the model.

Two things jump out. First, LRU is completely flat from 8 gigabytes to 64 gigabytes. An eightfold increase in capacity buys **nothing at all**.

Second, Belady over that same range climbs from 39 percent to 62 percent, which means the flatness belongs to the policy and not to the workload.

The lever is the policy, not the size: LRU is flat where Belady climbs (Created by [Fareed Khan](#))

CAVEAT, and it matters

This trace was recorded during a run that re-prefills the whole prefix every step. In this trace, the same experts are legitimately touched ~68 times. Steady-state incremental decoding has far less reuse, and its hit rates will be LOWER than this curve suggests. Treat these numbers as an upper bound on what caching can do, not a forecast.

At 64 gigabytes there is a **25.5 point gap** between what LRU achieves and what an optimal policy would achieve at exactly the same memory. That is the most interesting number in this section, because it says the promising direction is a better replacement policy rather than more RAM.

The file also carries its own warning, which turns out to be the important part.

The trace came from a run that recomputes the entire prefix at every step, so it manufactures reuse that steady-state decode does not have. The prediction is therefore an optimistic bound.

Keep that table in mind. In a few sections we are going to measure it directly, and the two do not agree.

Proving It: a Tiny Oracle First

We now have every component. Before running the released checkpoint, we need to know that the composition is right, and that is a different question from whether each kernel is right.

Four levels of proof, and only the last two touch the released checkpoint (Created by [Fareed Khan](#))

The rounding budget for 93 layers at hidden size 7168 (Created by [Fareed Khan](#))

The middle level is a tiny model with the same tensor graph. Thirteen layers, hidden size 128, vocabulary 256, built to have exactly the same structure as the full model.

Why thirteen layers and not five? Because attention residuals operate in blocks of twelve, and their failure modes cannot appear until two blocks are complete and a third is in progress. A five layer model would never exercise the boundary logic at all.

Op fixtures, a toy oracle, then the released checkpoint (Created by [Fareed Khan](#))

The oracle runs three gates.

```
3. full-model oracle gates on the 13-layer reference
checkpoint: 628 tensors loaded
layer map (0-based): KKKMKKKMKKKMM (M=MLA, K=KDA; dense layer = 0)
```

```
attn_res boundaries at: 0 3 6 9 12
prompt_ids 12, full_ids 32, tf_pred 32
all layer weights bound

GATE 1  teacher forcing : 32/32 positions match tf_pred
        generated span  : 20/20  <- must be exact
GATE 2  greedy decode   : 20/20 generated tokens match full_ids
GATE 3  incremental     : 20/20 generated tokens match full_ids  <- KV cache + ca

VERDICT: ENGINE MATCHES THE REFERENCE EXACTLY
```

Three different execution paths giving identical token ids. Teacher forcing checks the forward pass, greedy decode checks the sampling loop, and the incremental gate checks that the KV cache and the carried KDA state produce the same answer as recomputing from scratch. All three are exact, because on a discrete argmax there is no such thing as a small error.

And now the caveat that has to go directly underneath it.

```
The strongest-sounding line in gates.txt is about a toy model. "VERDICT: ENGINE
MATCHES THE REFERENCE EXACTLY" refers to the 13-layer, hidden-128, vocab-256 ora
- NOT the 2.8T checkpoint.
```

That is from the project's own limitations file, and it is right to say so loudly. "Matches the reference exactly" reads like the end of the story, and it is the end of a story about a nine megabyte fixture.

One more thing about how the fixtures were built, because it is the difference between a test suite and a decoration. Every fixture was designed to fail a specific plausible wrong implementation. The router fixture

reorders its top two experts on five of six rows, so an implementation that ignores the routing bias fails it.

The SiTU-GLU fixture spans inputs from 0.1 to 1000, because in the near-linear region the bounded tanh is indistinguishable from the identity and an implementation with the caps left out entirely would pass.

A test that a plausible bug would survive is not a test.

Proving It on the Full Checkpoint

Now the full model. There are two questions left. Is each of the 93 layers wired correctly, and does the whole stack produce the right numbers.

For the first, every layer is run in isolation and compared against PyTorch in three stages: the attention output, the router decision, and the MoE output.

```
# A wrong binding does not miss by 1e-6, it misses by ~1.  
tol = 1.2e-7 * math.sqrt(width) * 50
```

That runs for all 93 layers.

L87	KDA	PASS	3 stages	worst 0.00x budget	44s
L88	KDA	PASS	3 stages	worst 0.00x budget	46s
L89	KDA	PASS	3 stages	worst 0.00x budget	43s
L90	KDA	PASS	3 stages	worst 0.00x budget	45s
L91	MLA	PASS	3 stages	worst 0.00x budget	46s
L92	MLA	PASS	3 stages	worst 0.00x budget	49s

```
=====
```

LAYERS	0..92	93 passed, 0 failed	(69 KDA, 24 MLA)	5639 s total
--------	-------	---------------------	------------------	--------------

worst stage across all passing layers: 0.00x of its rounding budget

=====

VERDICT: ALL LAYERS CONFORM

All 93 layers checked against torch: 93 passed, 0 failed, 5639 seconds (Created by [Fareed Khan](#))

reference forward: 93 layers, 5 prompt ids, hidden 7168, vocab 163840
embedded 5 ids (BF16)

L0	KDA dense	20.7 s	h max 0.145457
L1	KDA MoE	44.7 s	h max 0.187096
L2	KDA MoE	41.8 s	h max 0.463652
L3	MLA MoE	39.2 s	h max 1.284419

Ninety-three layers, 69 KDA and 24 MLA, and the worst error across all of them rounds to zero percent of the allowed budget. Note the last two rows:

L91 and L92 are both MLA, which is the architectural quirk the config told us about at the very beginning, now visible in the verification output.

For the second question we need a full forward pass from an independent implementation. That is expensive.

```
L90  KDA  MoE      35.4 s  |h|  max  26.325483
L91  MLA  MoE      38.1 s  |h|  max  31.064180
L92  MLA  MoE      36.7 s  |h|  max  31.064180
```

```
final position: argmax token 2494, logit 15.021948, mean -0.462536, max 15.02194
total 3608.5 s
```

Three thousand six hundred seconds. One hour for a single forward pass over five tokens in PyTorch, loading and freeing one layer at a time because the fp32 expansion would be about 227 gigabytes. Our C engine did the same forward in 169.73 seconds.

Now compare all 163,840 logits, not just the argmax.

```
# argmax only cares which entry is biggest, so compare all 163,840 elementwise.
budget = 1.2e-7 * np.sqrt(hidden) * 50          # 5.1e-04 at hidden 7168
```

Compare all 163,840 logits, not just the winner (Created by [Fareed Khan](#))

```
===== ELEMENTWISE LOGIT COMPARISON =====
```

```
prompt cross-check      : both sides ran [3, 4, 5, 6, 7]
vocab                   : 163840
C argmax                : 2494 (logit 15.021946)
reference argmax        : 2494 (logit 15.021948)
top-10 overlap          : 10/10
max |diff|              : 7.867813e-06
relative to max|ref|    : 5.237545e-07 (budget 5.1e-04)
mean |diff|             : 1.231738e-06
correlation              : 1.0000000000
```

VERIFIED: the C engine's logits match the torch reference over the FULL 93-layer stack on the real checkpoint, elementwise, and the argmax token agrees.

Both sides pick token 2494. The largest disagreement anywhere in 163,840 values is **7.87 times ten to the minus six**, which is about a thousandth of the allowed budget, and the correlation prints as 1.000000000. That is as close to “the same computation” as two different implementations get.

And now the caveat, which the corpus states about itself and I am not going to bury.

The logit parity run used prompt ids 3,4,5,6,7, which decode to: \$%&'(
That is synthetic junk, not text.

That same run reports `KV cache 0.00 B`, so the KV path is not exercised by the parity check at all.

The prompt is five punctuation characters, and the run used full recompute, so the KV cache was never touched. It is agreement on one position of a meaningless prompt, which is strong evidence about the arithmetic and no evidence at all about output quality. So let us go and generate some text.

The First Tokens

Let us ask it something with an answer.

Prompt in, 93 layers, one argmax, one word out (Created by Fareed Khan)

```
prompt    ids : 1008,10484,318,15383,387
prompt    txt : The capital of France is
generated ids : 17374,20829,10,427,414,1008,606,142957
generated txt : Paris.",~+          "The Eiffel~
              (~ marks a newline)
```

The first two tokens are 17374 = ' Paris' and 20829 = '."', - the model answer the question CORRECTLY.

Paris. The first token out of a 2.78 trillion parameter model running on a CPU in a few gigabytes of RAM is the right answer.

The trailing quote and the “The Eiffel” continuation are worth a sentence, because they are not a defect. There is no chat template applied here and no instruction tuning in the loop, so this is a base model continuing text rather than answering a question.

It has decided it is inside a JSON list of sentences about France, and it is continuing that list. That is exactly what a base model does.

Then the same prompt was run under seven different hard memory ceilings.

cap	trunk	cache	s/token	token ids	decoded
96G	48G	40G	72.3430	17374,20829,10	Paris.",~+~
64G	32G	24G	67.8921	17374,20829,10	Paris.",~+~
48G	24G	16G	63.1822	17374,20829,10	Paris.",~+~
32G	12G	12G	63.4072	17374,20829,10	Paris.",~+~
24G	10G	7G	60.8162	17374,20829,10	Paris.",~+~
16G	6G	4G	68.4805	17374,20829,10	Paris.",~+~
12G	4G	2G	61.0702	17374,20829,10	Paris.",~+~

Every row that ran must show the SAME tokens. A differing row is a bug.

Seven memory ceilings, seven different speeds, one identical answer (Created by [Fareed Khan](#))

From 96 gigabytes down to 12, the same three tokens. As the corpus puts it, this is not merely the same ids at every budget, it is the same **right answer** from 96 gigabytes down to 12.

There is one run I want to flag rather than quote. A smoke test on the campaign machine reproduced the archived token ids exactly, which is good, but it recorded 97.26 seconds per token while the background package updater was competing for CPU. Those ids are evidence.

That timing is not, and it does not appear anywhere in this post as a performance number.

Sustained Generation: Text In, Text Out

Three tokens prove correctness. They do not prove the thing is usable. So here are four full generations, plain text in and plain text out, with the C tokenizer on both ends and no Python anywhere on the path.

One prefill wall, then a steady shelf, on every prompt (Created by [Fareed Khan](#))

All four run at `--trunk-gb 110 --cache-gb 13 --incremental`. That split is going to look strange when we get to it, because it gives almost everything to the trunk and almost nothing to the expert cache. I will show why it wins two sections from now.

Here is the loop that produces every output block in the rest of this post.

```
for (int g = 0; g < gen; g++) {
    k3_cache_reset_stats(&cache);
    const double ts = now_s();
    int frc;
    if (incremental) {
        /* step 0 feeds the whole prompt; later steps feed only the new token */
        const int base = w.cached;
        const int nT = (g == 0) ? np : 1;
        frc = forward(&w, &c, &cache, seq + base, nT, lg, sc, h, br, ks);
        w.cached = base + nT;
    } else {
        frc = forward(&w, &c, &cache, seq, T, lg, sc, h, br, ks);
    }
    /* Abort the run rather than argmax a buffer the forward never wrote. */
    if (frc != 0) {
```

```

        fprintf(stderr, "forward pass failed at generation step %d; aborting.\n"
        return 1;
    }
    const int nxt = argmax_(lg, c.vocab);
    const double dt = now_s() - ts;
    t_total += dt;
    const uint64_t req = cache.hits + cache.misses;
    printf("%-6d %-10d %-12.2f %-10.1f %-10.2f %.3f\n", g, nxt, dt,
        req ? 100.0 * cache.hits / req : 0.0,
        (double)cache.bytes_read / 1e9, 1.0 / dt);
    fflush(stdout);
    /* Roll the per-step figures up before the next reset wipes them. */
    expert_s_total += cache.load_seconds;
    expert_gb_total += (double)cache.bytes_read / 1e9;
    expert_reqs_total += cache.hits + cache.misses;
    expert_evict_total += cache.evictions;
    seq[T++] = nxt;
    outtok[nout++] = nxt;
}

```

The sampler is one line, and it is the only one there is.

```

static int argmax_(const float *v, int n)
{ int b = 0; for (int i = 1; i < n; i++) if (v[i] > v[b]) b = i; return b; }

```

Greedy, with no temperature and no top-p. That is a deliberate choice rather than an unfinished one, because greedy decoding is what makes the output identical at every memory budget, and that property is what most of the testing in this post depends on.

Here is the first run, trimmed to its shape.

```
tokenized: 17 bytes -> 5 ids
prompt    : 5 tokens, generating 24

trunk stream: 108.81 GB packed, 90/93 layers PINNED (108.19 GB), ring 1 x 1.29 G
              deterministic hit rate 96.8%
expert cache: 740 slots x 17.56 MB = 12.99 GB (0.90% of the 1.45 TB expert pool)
incremental decode: KV cache 70.96 MB for 24 MLA layers at 30 positions
```

STEP	TOKEN	SECONDS	CACHE HIT	READ GB	TOK/S
0	1040	53.04	100.0	89.21	0.019
1	149803	8.92	100.0	25.83	0.112
2	316	8.92	100.0	25.83	0.112
3	374	8.74	100.0	25.83	0.114
4	1491	8.85	100.0	25.83	0.113
5	261	8.91	100.0	25.83	0.112

```
-----
24 tokens in 255.8 s, 10.66 s/token average

--- generated text ---
Kelsey and I am a certified teacher. I have experience tutoring after school at
middle school level. I have taught
-----

PEAK RSS for the whole run: 127.89 GB  <- quote this, not the plan
```

The prompt was “**Hello! My name is**”, and the model answered “ **Kelsey and I am a certified teacher. I have experience tutoring after school at the middle school level. I have taught**”.

That is fluent, grammatical, and it holds a persona across the whole span. It invented a name and then stayed consistent with it for twenty-four tokens.

Look at the timing column, because that shape repeats in every run. Step 0 takes **53.04 seconds** and reads 89.21 gigabytes. Every step after it takes about **8.9 seconds** and reads exactly **25.83 gigabytes**.

The first step pays for the whole prompt, and then the cost per token flattens completely.

Prefill scales with the prompt, and nothing after it does (Created by [Fareed Khan](#))

One prefill wall, then a steady shelf, on all four prompts (Created by [Fareed Khan](#))

Now a harder prompt. Given `def fibonacci(n):` the model produced this.

--- generated text ---

```
if n <= 1:
    return n
else:
    return fibonacci(n-1) + fibonacci(n-2)
-----

28 tokens in 299.3 s, 10.69 s/token average
```

That is a **correct recursive Fibonacci implementation**. The base case `if n <= 1: return n` is right, the indentation is right, and the recurrence `fibonacci(n-1) + fibonacci(n-2)` is right and only cut off because the token budget ran out mid-expression.

And the fourth prompt is my favourite, because the model describes itself. Given “Kimi K3 is a mixture-of-experts language model. It works by” it continued:

```
--- generated text ---
routing each token through a small subset of its total parameters, which keeps
inference fast and cheap relative to its size. The model is trained on a large c
-----

32 tokens in 361.6 s, 11.30 s/token average
```

“routing each token through a small subset of its total parameters” is exactly what we spent the last twelve sections implementing. Kimi K3 activates 16 of 896 experts per layer, which is about 104 billion of 2.78 trillion parameters, and the description is accurate.

That last run also shows the prefill cost scaling. Its prompt is 17 tokens rather than 5, and its step 0 took **97.98 seconds and read 200.67 gigabytes**,

roughly double the five-token prompts, while every subsequent step still read exactly 25.83 gigabytes.

Prefill scales with the prompt, and nothing after it does (Created by [Fareed Khan](#))

Four prompts is a demonstration and not a benchmark, and I want to be clear about that. But the shape is consistent: about 10.7 to 11.8 seconds per token in sustained decode, at 127.9 gigabytes of peak memory, producing text that is grammatical, factual and in one case correct Python.

The Memory Ladder: 8 GB to 224 GB

Now the central experiment. We claimed at the start that this runs in 8 gigabytes and in 224 gigabytes with identical output. Here is how that was measured.

The important part is that the memory limit has to be **enforced**. Telling the engine to use 8 gigabytes on a 228 gigabyte machine measures nothing,

because nothing stops it from using more.

Cap the memory with a cgroup, then check the ids are identical (Created by [Fareed Khan](#))

```
# MemorySwapMax=0 matters as much as MemoryMax: without it an over-budget rung
# swaps instead of dying, and its s/token measures swap bandwidth.
systemd-run --scope --user -q \
  -p MemoryMax=${TOT}G -p MemorySwapMax=0 \
  ./bin/k3 "$MODEL" --ids "$IDS" --gen "$GEN" \
  --trunk "$TRUNK" --trunk-gb "$TR" --cache-gb "$CA" --incremental \
  --out "$OUT/$tag.json" > "$OUT/$tag.log" 2>&1
```

Twelve budgets, same prompt, same binary, eight tokens each. Here is the bottom of the ladder.

```
=== MEMORY LADDER ===
host   : 124 cores, 228 GB RAM
prompt : 1008,10484,318,15383,387   gen=8   incremental=yes
-----
RUNG 8 GB   trunk=2.5   cache=0.5
-----
32.69 s/token | hit 0.0% | 25.83 GB read | peak RSS 8.24 GB
ids: 17374,20829,10,427,414,1008,606,142957
```

8.24 gigabytes of peak resident memory. That is the number the fit ledger has been walking towards since the second section. A 2.78 trillion parameter

model, whose weights are 5.56 terabytes at bfloat16, generating text inside 8.24 gigabytes on a CPU.

And here is the whole table.

```
=== LADDER COMPLETE ===
```

total_gb	pin_layers	cache_gb	s_per_tok	trunk_hit	gb_read	peak_rss_gb
8	0	0.49	32.69	0.0	25.83	8.24
12	0	2.79	31.41	0.0	25.83	10.53
16	3	4.39	32.21	2.8	25.83	16.00
24	7	7.58	31.85	6.6	25.83	23.95
32	11	10.80	31.44	10.3	25.83	31.90
48	19	17.19	29.76	17.9	25.83	47.80
64	27	23.59	28.60	25.4	25.83	63.71
96	43	36.39	24.40	40.5	18.11	95.51
128	60	49.19	29.40	56.5	17.51	128.18
160	76	61.99	26.31	71.5	17.28	159.98
192	90	77.00	21.32	84.7	16.65	191.83
224	90	108.98	19.21	84.7	14.53	223.82

```
ids, every row: 17374,20829,10,427,414,1008,606,142957
```

Read that last line first. **Every single rung produced byte-identical output.** Twelve different memory budgets spanning a factor of 28, and the token ids are the same in every one. Memory buys speed here, and it does not buy capability.

One note on the `hit_pct` column, because it is easy to misread. That is the **trunk** hit rate, not the expert cache hit rate, and the two behave completely differently. We will come to the expert one in the next section.

28x the memory buys 1.70x the speed, and most steps are inside the noise (Created by [Fareed Khan](#))

Now the timing column. Going from 8 gigabytes to 224 gigabytes takes 32.69 seconds per token down to 19.21. That is **28 times the memory for 1.70 times the speed.**

If you are choosing hardware, the jump from 8 gigabytes to 64 gigabytes buys 14 percent. The memory is not where the speed is.

The memory plan is not lying: every rung lands on its own budget (Created by [Fareed Khan](#))

The peak RSS column is worth its own picture, because it is the evidence that the memory plan is honest. Ask for 8 gigabytes and it uses 8.24. Ask for 224 and it uses 223.82. Every rung lands on its own budget.

There is also a cold-start effect baked into these numbers that makes the high rungs look worse than they are.

Token zero pays the pinning cost, so short runs understate the hit rate (Created by [Fareed Khan](#))

At the top rungs 90 of 93 layers are pinned, so the steady-state trunk hit rate is 96.8 percent. The table says 84.7 percent because these are eight token runs and token zero pays the full pinning cost, so seven tokens each hit 90 layers out of 744 total binds. That is exactly 84.7 percent.

The sustained generations from the previous section, which ran 16 to 32 tokens, averaged 10.66 to 11.79 seconds per token against this table's best of 19.21, which is the same effect from the other direction.

The Cache That Was Not Participating

Now look at the `gb_streamed` column in that table again, because something is wrong with it.

```
8    GB budget -> 25.83 GB read per token
12   GB budget -> 25.83 GB read per token
16   GB budget -> 25.83 GB read per token
24   GB budget -> 25.83 GB read per token
32   GB budget -> 25.83 GB read per token
```

```
48 GB budget -> 25.83 GB read per token
64 GB budget -> 25.83 GB read per token
```

Seven consecutive budgets read **exactly** the same number of bytes. Over that range the expert cache grows from 28 slots to 1,344 slots, a factor of 48, and the bytes moved do not change by a single decimal.

The expert cache does nothing at all until about 36 GB of arena (Created by [Fareed Khan](#))

A cache that grows 48 times and changes the bytes moved by zero is not a cache that is working inefficiently. It is a cache that is **not participating at all**.

Remember the simulation from earlier, which predicted a flat 36.24 percent hit rate over exactly this range. The measurement says the flat value is not 36 percent, it is zero.

The simulation was wrong in both directions (Created by [Fareed Khan](#))

The simulation was wrong in both directions. It predicted flat 36.24 percent with a knee at 128 gigabytes, and the reality is flat 0.0 percent with a knee at about 36 gigabytes of arena. Its own caveat had warned us: the trace came from full-recompute decode, which touches each expert about 68 times and manufactures reuse that steady-state decode does not have.

Why is the cache so useless? The answer is in the model, not the engine. Remember Quantile Balancing from the routing section, the training technique whose whole purpose is to flatten expert usage across the pool.

Flat usage is precisely what defeats a least-recently-used cache. With no hot subset, a few gigabytes of arena retain nothing worth keeping. The weak caching is a property of Kimi K3, not a defect in our implementation.

That leaves a measurement problem, and it is a good one. Here is what the engine printed about its own cache during a generation run.

```
cache [final step]
  slots      : 740 of 17.56 MB = 12.99 GB arena (740 resident, 0 pinned)
  requests   : 1472 hits 1472 (100.00%) misses 0 evictions 1032
  of those hits : 38940 came from the batch prefetch, i.e. read from disk
                  this token; TRUE resident hit rate 0.00%
  read from disk: 25.83 GB in 5.06 s (5104 MB/s while loading)
```

A hit rate of 100.00 percent sitting two lines above a resident hit rate of 0.00 percent, for the same 1,472 requests. Both are measuring something and neither is measuring what we want.

One counter, three answers, and only one of them means avoided I/O (Created by [Fareed Khan](#))

Three definitions of one number, and only the third agrees with the bytes (Created by [Fareed Khan](#))

The first counts a request satisfied from the arena, but the batch prefetch put the expert there microseconds earlier by reading it off disk, so it reads about 100 percent at every cache size and tells you nothing about avoided I/O. The second counts only experts that were already resident before the step began, which is the quantity we actually care about, but it is measured against a window that resets every step.

The number that survives is derived rather than counted.

The only expert-cache metric that agrees with the bytes actually read (Created by [Fareed Khan](#))

An expert that had to be evicted is one that was not retained. At the 36.39 gigabyte rung that gives $1 - 1032/1472 = 29.89\%$ retained, against $18.11/25.83 = 70.1\%$ of the bytes still being read. Those two agree to the decimal, computed from completely different counters, which is what makes the derivation trustworthy.

And once the cache does engage, you can watch it happen step by step.

What a working expert cache actually looks like, step by step (Created by [Fareed Khan](#))

At the 8 gigabyte budget every step reads 25.83 gigabytes, flat forever. At 96 gigabytes the reads fall from 99.70 to 23.58 to 19.90 to 14.07 across the first four steps as the arena fills with experts that get used again.

That declining curve is what a working cache looks like, and it does not appear anywhere below about 36 gigabytes of arena.

Allocation Beats Capacity

So if the expert cache does nothing below 36 gigabytes, where should the memory go instead?

The arithmetic is not close. The trunk is 108.81 gigabytes and it is re-read in full on every single token. The experts are 25.83 gigabytes per token.

That is **4.2 times the bytes** and **2.9 times the time**.

The trunk is re-read in full every token, the experts are only sampled (Created by [Fareed Khan](#))

What one pinned layer removes from the guaranteed traffic (Created by [Fareed Khan](#))

A gigabyte given to the trunk pins roughly one more layer and removes about 1.17 gigabytes per token of traffic that would otherwise happen with certainty. A gigabyte given to the expert cache removes, below the knee, nothing measurable.

Give the trunk everything until it is pinned, then feed the cache (Created by [Fareed Khan](#))

So we tested it directly. Fix the total memory, vary only the split. The harness exists for exactly this reason, because the ladder confounds two variables.

```
# memory-ladder.sh varies the total and cannot separate "more memory" from
# "better-allocated memory"; this script varies only the allocation, so any diff
# it finds is attributable to the split alone.
#
# Fractions of the budget given to the trunk, cache-heavy first so the sweep runs
# AGAINST the hypothesis rather than with it.
FRACS="0.10 0.25 0.60 0.86"
```

Here is the sweep at a fixed 128 gigabytes.

total_gb	trunk_gb	cache_gb	s_per_tok	hit_pct	gb_read	peak_rss
128	12.3	110.7	28.38	44.02	14.46	127.89
128	30.8	92.2	25.20	42.93	14.74	127.53

128	49.2	73.8	25.69	33.97	17.06	128.14
128	73.8	49.2	18.37	32.20	17.51	128.19
128	98.4	24.6	19.46	0.0	25.83	127.36
128	110.0	13.0	16.80	0.0	25.83	127.85

At a fixed budget, giving the trunk more is 1.69x faster (Created by [Fareed Khan](#))

From 28.38 seconds per token down to 16.80, at **identical total memory**. That is 1.69 times faster from allocation alone, a 69 percent difference, and we will see shortly that it is twice the noise floor so it holds.

Now look at that table again, because it contains a result that runs the wrong way. The **fastest** configuration reads 25.83 gigabytes per token with a 0.0 percent cache hit rate. The **slowest** reads only 14.46 gigabytes with a 44.02 percent hit rate.

The winner reads 79% more expert bytes and still wins (Created by [Fareed Khan](#))

The winner moves 79 percent more expert bytes than the loser and beats it by 69 percent anyway. That is the cleanest possible statement that the expert cache is not where the time goes. Optimising the number everyone would naturally optimise, the cache hit rate, actively takes you to the slower configuration.

So the rule is simple: give the trunk everything until it is fully pinned, and only then feed the expert cache.

I have to be honest about how strong this result is. The trend is not perfectly monotonic, and there are two inversions inside the 128 gigabyte series and one at 32 gigabytes, all of them well inside the noise.

The claim rests on the rank correlation across twelve points at two different totals, which is minus 0.886 and minus 0.714, plus a mechanism that predicts it. The direction is solid. The precise figure of 1.69 has not been replicated three times, so treat the magnitude as approximate and the direction as established.

Measuring the Measurement

Everything in the last three sections is a timing, so before quoting any of them we should establish how repeatable a timing on this machine actually is. Three runs of one identical configuration, back to back on a quiet machine, answer that.

```
[09:36:39] --- replicate 1/3 (trunk 110 / cache 13, gen 8, identical every time)
      8 tokens in 118.2 s, 14.78 s/token average
[09:39:14] --- replicate 2/3 ---
      8 tokens in 117.3 s, 14.67 s/token average
[09:41:22] --- replicate 3/3 ---
      8 tokens in 161.1 s, 20.14 s/token average

=== summary ===
rep 1: 14.78 s/token average
rep 2: 14.67 s/token average
rep 3: 20.14 s/token average
```

Same binary, same prompt, same flags, same machine, same minute (Created by [Fareed Khan](#))

Three identical runs, so a smaller difference is not an effect (Created by [Fareed Khan](#))

Same binary, same prompt, same flags, same machine, same minute. Mean 16.53 seconds per token, standard deviation 3.13, spread **33.1 percent**. A third of the measurement is noise.

That number is the bar every timing claim in this post has to clear, and it rules a line through several of the smaller steps in the ladder. The 12

gigabyte rung sitting four percent under the 8 gigabyte rung is not an effect. The 11 percent trunk-first improvement at a 32 gigabyte total is not an effect.

The dip at 128 gigabytes is not an outlier, it is dispersion.

Two results clear the bar comfortably. The ladder spans **70 percent** from end to end, and trunk-first at 128 gigabytes is **69 percent**. Both are more than twice the floor.

Only two timing effects clear the band (Created by [Fareed Khan](#))

And every non-timing result is untouched, because counts and byte totals are not stopwatch readings. The byte-identical output across twelve budgets, the flat 25.83 gigabytes across seven rungs, the 0.0 percent retention from 28 slots to 1,344, the trunk hit rate tracking the pinned fraction: no amount of scheduling jitter moves any of those.

So where does a third of a measurement go? The engine moves about 135 gigabytes per token, so it is dominated by storage, and storage on this machine is not a constant. Two runs of the same configuration, taken from two different sweeps, show it directly.

run A (from the ladder)	run B (from the <code>split</code> sweep)
trunk (STREAMED) 73.80 GB	trunk (STREAMED) 73.80 GB
expert cache 49.20 GB	expert cache 49.20 GB
60/93 layers PINNED (72.34 GB)	60/93 layers PINNED (72.34 GB)
2802 slots	2802 slots
requests 1472 evictions 998	requests 1472 evictions 998
binds 744, hits 420 (56.5%)	binds 744, hits 420 (56.5%)
<code>read</code> 374.99 GB <code>in</code> 138.40 s	<code>read</code> 374.99 GB <code>in</code> 63.84 s
(2709 MB/s)	(5874 MB/s)
29.40 s/token	18.37 s/token

They pinned the same layers, allocated the same slots, made the same number of requests, evicted the same number of experts and read **exactly the same 374.99 gigabytes**. Every count matches.

The only thing that differs is that one of them got 2,709 megabytes per second out of the disk and the other got 5,874, a factor of 2.17 on identical work, and so one recorded 29.40 seconds per token and the other 18.37.

The same configuration, measured twice, doing identical work (Created by [Fareed Khan](#))

That is the mechanism, measured rather than guessed. The variance is not scheduling jitter or NUMA placement, it is the device.

Storage Is the Whole Game

Which leads to the rule that governs every performance number here. This engine is I/O bound at every budget, so a difference in storage dominates almost any difference in code.

This is an I/O problem at every budget, from 8 GB to 224 GB (Created by [Fareed Khan](#))

Both terms are whole-run totals, and this is an I/O problem at every budget (Created by [Fareed Khan](#))

Between 40.9 and 60.6 percent of wall clock is spent waiting on the disk, at every single rung. So the same binary on two different devices is two different engines.

The same code on two devices, and the gap is the device (Created by [Fareed Khan](#))

That is worth stating plainly because it changes what a benchmark means. A run at 31.71 seconds per token on local NVMe and a run at 70.62 on a network volume differ by 2.2 times, and none of that difference is attributable to anything in the source.

Any comparison between two configurations has to hold the device fixed, or it is measuring the device.

It also means the machine check that ships with the engine has to measure storage, and that the tool measuring it properly cannot use `dd`.

```
# dd is one sequential stream at queue depth 1. The expert path is random
# 17.55 MB O_DIRECT reads, up to 16 outstanding. On network storage they diverge
SZ = 17547264    # one real Kimi K3 routed expert, to the byte
N  = 40         # reads per measurement
for qd in (1, 4, 16):
    measure(path, qd)
```

Now, one honest optimisation result. Switching the arenas from 4 kilobyte pages to 2 megabyte hugepages was measured as a clean A and B on one binary, using an environment variable rather than two builds.

```
##### A: 4 KB pages (previous behaviour)
4 tokens in 45.5 s, 11.37 s/token average
read 153.02 GB in 24.36 s (6282 MB/s)

##### B: 2 MB hugepages (new)
4 tokens in 43.0 s, 10.76 s/token average
read 153.02 GB in 22.64 s (6758 MB/s)

=== output equality (a speedup that changes tokens is worthless) ===
A: [161427, 11294, 58776, 123595]
B: [161427, 11294, 58776, 123595]
IDENTICAL
```

The tokens are identical, which is the first thing to check about any optimisation. And the timing improved by 5.4 percent, which is comfortably **inside the 33 percent noise floor**, so I am not going to claim it as a win. The mechanism is sound and the measurement does not establish the size.

The reason it is one binary and not two is stated in the code, and the same argument appears independently in three different files.

Why We Do Not Quantize the Trunk

There is an obvious optimisation we have not done, and its absence is deliberate rather than an oversight.

The trunk is 108.81 gigabytes of bfloat16. Quantizing it to int8 would halve that, and to int4 would quarter it. Every other engine of this shape offers a bit-width knob.

This one has exactly two weight types and no knob at all.

```
enum { K3_WF32 = 0, K3_WBF16 = 1 };
```

The reason is that we measured what it would cost. The study sampled 31 attention tensors from the released checkpoint over HTTP range reads, 384 rows each, and quantized them with symmetric per-row scaling.

Symmetric per-row quantization, the method behind the int8 and int4 study (Created by [Fareed Khan](#))

type	tensor	int8 mean	int4 mean	ratio
KDA	L13.o_proj	0.01188	0.21122	17.8x
MLA	L3.kv_b_proj	0.00736	0.13355	18.2x
MLA	L11.o_proj	0.01399	0.24612	17.6x
MEAN over	KDA-layer tensors	0.01046	0.18746	17.9x (n=2)
MEAN over	MLA-layer tensors	0.00948	0.17154	18.1x (n=18)
MEAN over ALL	sampled tensors	0.00961	0.17383	18.1x (n=31)

int8 costs about 1% mean relative error on these weights; int4 costs about 17%.

int8 costs about 1% mean relative error on these weights; int4 costs about 17%.

No layer type tolerates 4 bits any better than the others (Created by [Fareed Khan](#))

Int8 costs about one percent and int4 costs about seventeen, a ratio of 18 that holds across every tensor sampled. And the mean understates the damage, because the tail is much worse.

int8 costs about 1%, int4 costs about 17%, and its worst rows cost 65% (Created by [Fareed Khan](#))

The worst individual rows at int4 reach 45 percent, 56 percent and **65 percent** relative error. Those are not rounding artefacts, those are different numbers.

There is also a strong hint from the model authors themselves. The technical report says the experts are MXFP4 with quantization-aware training, “while all non-expert components remain in higher precision”. That list of non-expert components is exactly this trunk.

It was deliberately not quantized, and it was never trained to tolerate four bits.

So the engine faces a choice for those 108.81 gigabytes: quantize it and lose accuracy, or stream it and lose only time. It streams it, byte for byte.

Seconds are bought back with RAM, and rounding error is not, at any budget (Created by [Fareed Khan](#))

That is the whole argument in one line. A lossless stream costs seconds per token, and those seconds are recoverable by giving the engine more RAM, which is exactly what the memory ladder demonstrated. Accuracy lost to four-bit rounding is not recoverable at any budget.

Four caveats on that study, all of which the file states about itself. This is weight reconstruction error and not output quality, and a 17 percent weight error does not straightforwardly become a 17 percent worse model. It sampled 384 rows per tensor rather than whole tensors.

It covered attention projections only, with no MoE, no embeddings and no output head. And no downstream logit or token comparison was ever run at int4, so the quality cost is bounded rather than measured.

What This Engine Does Not Do

Scope, stated plainly.

- **No chat template.** Base model continuations, not replies.
- **Greedy decoding only.** No temperature, no top-p, no top-k. Greedy is what keeps the output identical across budgets.

- **No chunked prefill.** The ceiling is 32,768 tokens, but a 21,000 token prompt is one quadratic pass and does not finish.
- **No million-token context.** That is a memory fact, not an engine limit.

The advertised million-token context is a memory fact, not an engine limit (Created by [Fareed Khan](#))

- **No vision.** MoonViT-V2 is fully specified in `config.json` at 27 layers, and has zero code.
- **No SIMD in the KDA recurrence.** The matmuls have AVX2 paths. The recurrence is still scalar C.
- **No quality benchmark.** No perplexity, no task eval. At 11 seconds per token that is days of compute, and it would measure Kimi K3 rather than this engine.

What this engine does not implement, drawn to scale (Created by [Fareed Khan](#))

That absent vision encoder is 0.057 percent of the checkpoint and downloadable on its own, which makes implementing it a 0.9 gigabyte job rather than a 1.56 terabyte one.

Running It End to End

If you want to run this yourself, here is the whole path.

From a 1.56 TB checkpoint to the right answer, on one machine (Created by [Fareed Khan](#))

First, ask the machine whether it can. The doctor script checks the toolchain, the instruction set, the memory and the storage, and then names

a preset.

```
# The preset boundaries follow the measured memory ladder.
# Note this keys on MemAvailable, not MemTotal.
if [ "$AVAIL_GB" -ge 192 ]; then PRESET=server;      EXPECT=~19-21 s/token"
elif [ "$AVAIL_GB" -ge 96 ]; then PRESET=workstation; EXPECT=~24 s/token"
elif [ "$AVAIL_GB" -ge 32 ]; then PRESET=desktop;   EXPECT=~28-31 s/token"
elif [ "$AVAIL_GB" -ge 10 ]; then PRESET=laptop;    EXPECT=~32 s/token"
else PRESET=""; fi

if [ -n "$PRESET" ]; then
    ok "recommended preset: --preset $PRESET    (expect $EXPECT)"
else
    bad "under 10 GB available - below the engine's floor (~8.2 GB measured peak"
fi
```

Then build and test, which needs no weights at all and takes seconds.

```
./scripts/k3-doctor.sh
make -j
make test
```

Then get the model. It is 1.56 terabytes and you want about 1.7 to leave room for the packed trunk.

```
export HF_TOKEN=...
./scripts/download-model.sh /models/k3
./scripts/pack-trunk.sh /models/k3 /models/k3trunk
```

The download script verifies the shard count and the exact byte total, and then every individual shard size, because a partial download does not fail loudly. It produces wrong tokens.

Checking per shard turns “re-download 1.56 terabytes” into “re-download this one 17 gigabyte file”, and it catches the one case a total cannot, which is two shards wrong in opposite directions by the same amount.

The presets themselves are four numbers each.

```
/* trunk GB, cache GB. The boundaries come from the measured ladder, and `server`
 * the best value: it pins 90 of 93 trunk layers, which is where the throughput
static const K3Preset K3_PRESETS[] = {
    { "laptop",      3.0,  1.0 }, /* ~8.2 GB peak RSS, ~32 s/token */
    { "desktop",    16.0, 10.0 }, /* ~31.9 GB, ~31 s/token */
    { "workstation", 60.0, 30.0 }, /* ~95.5 GB, ~24 s/token */
    { "server",     110.0, 13.0 }, /* ~128 GB, ~17 s/token */
    { "max",        110.0, 109.0 }, /* ~224 GB, ~19 s/token */
};
```

Note that `max` is not faster than `server` in our measurements. The extra 96 gigabytes buys nothing outside the noise floor, which is the trunk-first rule showing up one more time.

And then run it.

```
./bin/k3 /models/k3 \
  --trunk /models/k3trunk --preset server \
  --tok /models/k3 \
```

```
--prompt "The capital of France is" \  
--gen 16 --incremental
```

That is the whole thing. A 176 kilobyte binary, one packed file, and a model with 2.78 trillion parameters answering a question on a CPU.

Let us close the ledger we opened at the beginning. Every parameter at bfloat16 is 5.56 terabytes. The experts already ship at 0.53125 bytes per weight, which takes it to 1.56 terabytes on disk.

Routing means only 16 of 896 experts fire per layer, so 1.45 terabytes of that never needs to be in memory at all, leaving 113.49 gigabytes. Streaming the trunk a layer at a time turns that last floor into a dial, and the dial goes down to a measured **8.24 gigabytes**.

The point was never the speed. At 8 gigabytes it takes about half a minute per token, and pretending otherwise would be silly.

The point is that the model fits, that it produces the same tokens at 8 gigabytes as it does at 224, and that the gap between “you need a datacenter” and “you need a desktop” was four decisions about where bytes live rather than any change to the model itself.

Wanna chat about this? Reach me on [my LinkedIn](#).

AI +

Artificial Intelligence ✓

Data Science ✓

Machine Learning ✓

Python ✓



Published in Level Up Coding

Follow

351K followers · Last published 1 day ago

Coding tutorials and news. The developer homepage gitconnected.com && skilled.dev && levelup.dev



Written by Fareed Khan

Follow

76K followers · 0 following

I write on AI, <https://www.linkedin.com/in/fareed-khan-dev/>

